RunbookFX: Type- and Effect-Safe LLM Synthesis for Executable Incident Diagnosis and Mitigation

YIFAN XIAO, Peking University, China
SHIJIE LI, China Southern Power Grid Company Limited, China
YUHAO GE, China Southern Power Grid Company Limited, China

Large language models are increasingly deployed as autonomous agents for cloud incident response, yet their direct use admits hallucinated diagnoses, unauthorized actions, irreversible changes, and unauditable decision trails. We present RunbookFX, a typed functional domain-specific language that elevates incident response from natural-language suggestions to executable programs whose safety is established statically. The key insight is that incident-response safety decomposes into three interacting dimensions: risk severity, exercised capabilities, and rollback resource availability. RunbookFX formalizes this decomposition as a product effect algebra $\mathsf{Risk} \times \mathcal{K} \times \mathbb{N}$ whose four cross-component interaction axioms yield domain-specific safety theorems unexpressible in flat effect frameworks; a strong handler parametricity result then transfers these guarantees from a replay handler to any bisimilar live handler, bridging offline verification and production deployment. An LLM proposes candidate programs that a CEGIS-style verifier filters by static type checking and dynamic contract replay. A ~2,200-line Coq development discharges the product effect algebra, its composition-preservation property, and four core safety theorems: Effect WF Preservation, Progress, single-step No Unauthorized Action, and Rollback Linearity. Of the 27 supporting obligations in the substitution and multi-step layers, 18 now close with `Qed`—including all Canonical Forms, all effect-operation Inversion lemmas, Value Typing, de Bruijn weakening, and the typing-respecting reduction cases for `observe`, `act`, `rollback`, and the affirmative guard; the remaining nine trace back to the de Bruijn substitution lemma, whose proof skeleton follows Pierce et al. [2019]. Evaluated on RCAEval for root cause analysis and ITBench for end-to-end mitigation, RunbookFX achieves 64% Top-1 RCA accuracy against 53% for the best LLM baseline and 38% mitigation success at 3.3× the official ITBench agent, with zero safety violations and 100% rollback coverage by construction.

CCS Concepts: • **Software and its engineering → Language types**; *Domain specific languages*; • **Theory of computation → Type theory**.

Additional Key Words and Phrases: Algebraic effects, affine types, effect systems, program synthesis, LLM safety, incident management

ACM Reference Format:
Yifan Xiao, Shijie Li, and Yuhao Ge. 2026. RunbookFX: Type- and Effect-Safe LLM Synthesis for Executable Incident Diagnosis and Mitigation. *Proc. ACM Program. Lang.* 10, ICFP, Article 277 (August 2026), 34 pages. https://doi.org/10.1145/3828675

1 Introduction

Cloud-native systems increasingly rely on large language models for incident management, from root cause analysis to automated mitigation [Chen et al. 2024; Shetty et al. 2024]. Recent work demonstrates that LLM agents equipped with tool-use capabilities can diagnose faults by querying

Authors' Contact Information: Yifan Xiao, Peking University, Beijing, China, 2401110754@stu.pku.edu.cn; Shijie Li, China Southern Power Grid Company Limited, Guangzhou, China, geyh@csg.cn; Yuhao Ge, China Southern Power Grid Company Limited, Guangzhou, China, pengqq@csg.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2475-1421/2026/8-ART277
https://doi.org/10.1145/3828675

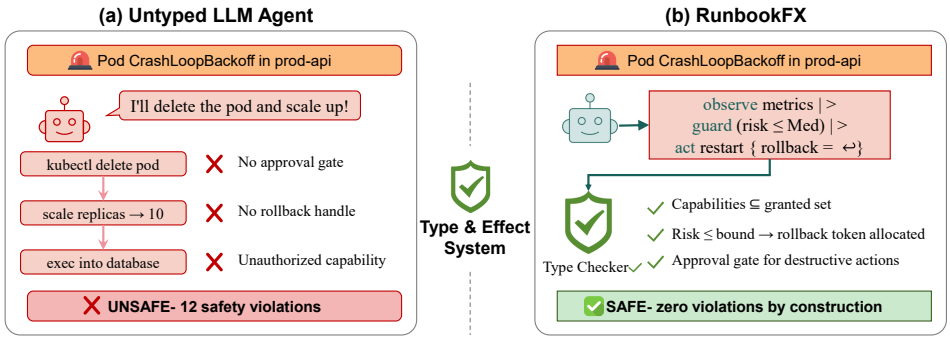


Fig. 1. Incident response on a representative Pod `CrashLoopBackOff` scenario. Bottom badges report aggregate counts over the 102-scenario ITBench SRE benchmark (Table 1): 12 safety violations for the untyped baseline; zero for RunbookFX. (a) An untyped LLM agent translates reasoning directly into API calls, producing unauthorized deletions, unrecoverable scaling, and unapproved database access. (b) RunbookFX constrains the LLM to synthesize a typed DSL program; the type-and-effect system verifies capability authorization, allocates rollback tokens for risky operations, and enforces approval gates, rejecting unsafe candidates *before* execution.

logs, metrics, and traces, and can execute remediation actions such as pod restarts, configuration rollbacks, and capacity scaling [Fu et al. 2025; Jha et al. 2025]. Figure 1 makes the gap visible: deploying such agents in production raises safety concerns beyond what current approaches address.

Existing LLM-based incident response systems operate as *untyped interpreters*: they translate natural-language reasoning directly into API calls without static guarantees about authorization, reversibility, or audit completeness. This design admits failures ranging from merely wasteful, such as querying irrelevant telemetry, to catastrophic: executing destructive operations without rollback capability, bypassing approval workflows, or acting on hallucinated evidence. The ITBench benchmark [Jha et al. 2025] reports that even the strongest LLM agents resolve only 11.4% of site reliability engineering (SRE) scenarios, and many failures stem not from incorrect diagnosis but from unsafe or malformed mitigation actions.

We decompose incident management into two complementary tasks (§2): *root cause analysis*, which produces ranked hypotheses with evidence chains from telemetry data, and *mitigation*, which produces executable remediation programs with rollback handles that clear triggering alerts while maintaining operational safety.

The incident response workflow of diagnose, verify, plan, execute, audit has a natural computational structure well-suited to a *functional programming* treatment. Diagnosis steps are pure observations that compose monadically; mitigation actions are effectful operations whose side effects should be tracked in the type system; and rollback tokens are resources whose consumption discipline is naturally captured by affine types. This observation motivates RunbookFX, a typed functional DSL for incident diagnosis and mitigation. Rather than allowing LLMs to execute arbitrary tool calls, RunbookFX requires all incident responses to be expressed as *well-typed programs* in our DSL. The LLM shifts from high-risk production operator to constrained program synthesizer, generating candidate runbook programs that must pass static and dynamic verification before execution.

Three technical pillars hold this design together. *(i)* A product effect algebra Risk × $\mathcal{K}$ × $\mathbb{N}$ tracks risk severity, exercised capabilities, and rollback obligations, with four cross-component interaction axioms (I1–I4 in §5) that enable domain-specific safety theorems unexpressible in a flat product. *(ii)* A *handler bisimulation* relation, central to §6.4, decouples safety reasoning from concrete handler implementations: a program verified against a replay handler is automatically safe under any bisimilar handler, including production execution, because the bisimulation requires structural agreement on operations and rollback scope but *not* agreement on return values. This is what lets us verify offline against recorded telemetry while deploying online against live infrastructure. *(iii)* A counterexample-guided synthesis loop (CEGIS-style, §7) recasts each verification failure—a type error, capability violation, or runtime contract breach—as structured repair guidance fed back to the LLM. The formal safety guarantees of the type system make the loop sound: a candidate is executed only after it has passed both static and dynamic verification, so synthesis failures degrade to escalation rather than unsafe actions.

Contributions. This paper makes the following contributions:

- **(C1)** A typed functional DSL grounded in a product effect algebra Risk × $\mathcal{K}$ × $\mathbb{N}$ with four *interaction axioms* that couple the components, yielding three domain-specific safety theorems mechanized in Coq at ∼2,200 lines. Bounded effect polymorphism enables generic reusable combinators while preserving decidable type checking (§4–5).
- **(C2)** A *handler bisimulation* relation $H_1 \sim_\kappa H_2$ and a *strong handler parametricity* theorem establishing that safety properties verified under one handler transfer to any bisimilar handler. The compositionality result rests on value-agnostic bisimulation for algebraic effect systems (§6).
- **(C3)** A CEGIS-style synthesis pipeline with a soundness theorem: verified candidates satisfy the type system's safety invariants under any well-formed handler (§7).
- **(C4)** Evaluation on RCAEval and ITBench, achieving 64% Top-1 RCA accuracy against 53% for the best baseline and 38% mitigation success at 3.3× the official baseline, with zero safety violations (§8).

§2 formalizes the two tasks; §3 sketches the system architecture. §4–6 present the DSL, type system, and semantics. §7 describes the synthesis pipeline; §8 reports experiments. §10 discusses related work and §11 concludes. A representative case study appears in Appendix E; two further case studies appear in the supplementary material.

2 Task Definition

We decompose incident management into two complementary tasks that RunbookFX addresses within a unified framework.

2.1 Task A: Root Cause Analysis (RCA)

Given incident telemetry comprising metric time series **M**, log streams **L**, distributed traces **T**, and an optional service topology graph G, the RCA task requires two outputs: first, a ranked list of root cause hypotheses identifying the faulty component and failure type; second, an *evidence chain* consisting of concrete diagnostic observations that support each hypothesis. We require that evidence chains be grounded in actual query executions rather than natural-language assertions, ensuring every diagnostic conclusion is backed by observable data.

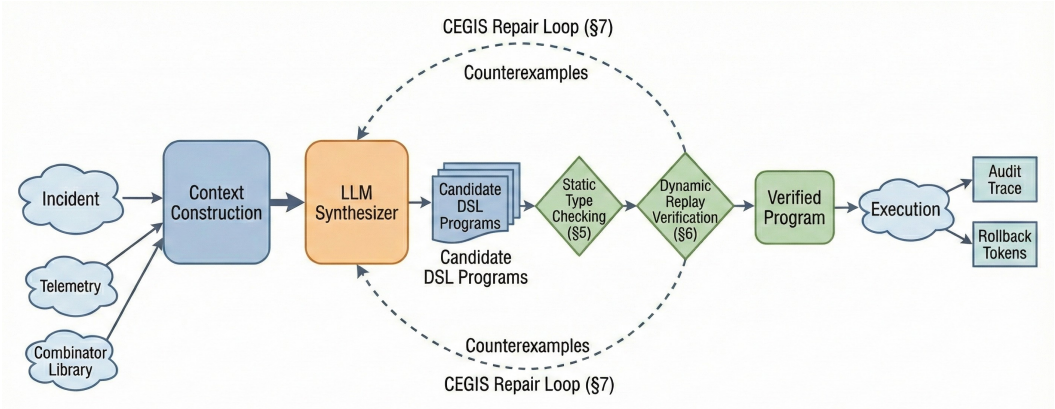


Fig. 2. The RunbookFX pipeline. An incident triggers context construction from telemetry and a combinator library. The LLM synthesizes candidate DSL programs, which undergo static type checking (§5) and dynamic replay verification (§6). A verification-guided repair loop (dashed arrows, §7) feeds structured failure reports back to the synthesizer. The verified program executes with a full audit trace and rollback tokens.

2.2 Task B: Incident Mitigation

Given an incident description, optionally augmented with RCA output from Task A, the mitigation task requires three outcomes: an executable remediation program that may branch on runtime conditions, include idempotency checks, and carry rollback handles for every state-changing action; successful execution that clears the triggering alerts and restores service health; and compliance with safety constraints, including no unauthorized actions, no risk-threshold violations, explicit approval for disruptive operations, and complete audit trails.

2.3 Success Criteria and Failure Modes

A successful incident response satisfies three properties simultaneously: *diagnostic correctness*, where the identified root cause matches ground truth; *mitigation effectiveness*, where alerts are resolved and service recovers; and *operational safety*, encompassing zero capability violations, zero unapproved disruptive actions, and full rollback coverage.

We identify four failure modes that plague existing approaches. First, incorrect root cause identification leads to misguided remediation. Second, syntactically valid but semantically ineffective mitigation actions waste response time. Third, unauthorized or excessively risky operations such as restarting a database without backup verification cause avoidable harm. Fourth, irreversible actions lacking rollback capability leave the system in an unrecoverable state. RunbookFX's type system is specifically designed to prevent the third and fourth failure modes statically, while the synthesis pipeline with verification addresses the first and second dynamically.

3 System Overview

Figure 2 illustrates the RunbookFX pipeline. Given an incident alert, the system first assembles a synthesis context from the alert payload, telemetry, service topology, and a DSL combinator

library. The LLM then generates k candidate programs as DSL ASTs, each undergoing two verification stages: static type checking for capability authorization, risk compliance, and rollback availability (§5), followed by dynamic replay verification against sandboxed telemetry (§6). Failing candidates enter the verification-guided repair loop (§7); the highest-ranked verified candidate executes against the real environment with a full audit trace.

4 The RunbookFX DSL

This section presents RunbookFX, a functional domain-specific language designed to express incident diagnosis and mitigation workflows as composable, type-safe programs.

4.1 Design Goals

RunbookFX is guided by four design principles drawn from both the functional programming tradition and operational best practices. *Composability*: runbooks are built from reusable combinators that can be sequenced, branched, and nested, enabling modular construction of complex workflows from simple building blocks. *Executability*: every construct in the language has a concrete operational interpretation that interfaces with real infrastructure tools (log aggregators, metric stores, tracing systems, orchestration APIs). *Auditability*: the runtime semantics produce a structured trace for every execution step, recording inputs, outputs, evidence dependencies, and timing information. *Reversibility*: every state-changing action must produce a rollback handle, and the type system enforces that these handles remain available.

These four principles intentionally constrain expressiveness in favor of statically verifiable safety. RunbookFX replaces general recursion with bounded **retry**, replaces unchecked exceptions with typed rollback, and admits parallel composition only for read-only operations. Each choice is deliberate: bounded constructs guarantee termination and enable complete audit traces, while restricted concurrency ensures confluent interleaving in §6.2. The resulting language is expressive enough for the incident response domain, as the 837-scenario evaluation demonstrates, while admitting decidable type checking and sound Coq mechanization, presented in §5.

4.2 Core Syntax

Figure 3 presents the core syntax of RunbookFX in BNF notation.

Diagnosis combinators. The $\textbf{observe}_c$ family queries three telemetry sources: $\textbf{observe}_{\mathsf{read_logs}}$ (log retrieval), $\textbf{observe}_{\mathsf{read_metrics}}$ (metric time series), and $\textbf{observe}_{\mathsf{read_traces}}$ (distributed traces). Each observation requires the corresponding read capability and produces only ReadOnly effects. The **hypothesize** and **validate** combinators are not value constructors but *audit-commitment points*: **hypothesize** records that a particular evidence chain has been promoted to a candidate root-cause label, and **validate** records that an explicit boolean confirmation step was performed against fresh observations before any state change. Carrying unit-like payload types makes them syntactically distinct from raw data and forces them to appear at fixed positions in the trace; the type system then guarantees no **act** executes without a preceding **hypothesize**/**validate** pair when **guard** is conditioned on the validation result. This separates the runbook's epistemic claims ("I think the cause is X, and I verified it") from its data flow, and is what enables the type system to reject the "act first, diagnose later" pattern common in agent baselines.

Mitigation combinators. The $\textbf{act}_c$ construct executes a state-changing operation such as pod restart, configuration update, or capacity scaling that requires capability c and produces at minimum a SafeChange effect along with an *affine* $\mathsf{RollbackToken}^{\mathsf{aff}}$. A *rollback handle* in RunbookFX denotes a stored, capability-specific compensating action: for a configuration update it records

τ	$::=$	Unit | Bool | String | Incident	(base types)
	|	LogStream | MetricSeries | TraceGraph	(telemetry)
	|	Evidence | Hypothesis | Topology	(diagnostic)
	|	$\text{RollbackToken}^{\text{aff}}$ | List τ | $\tau_1 \times \tau_2$	(compound; aff = affine)
	|	$\tau_1 \xrightarrow{\varepsilon} \tau_2$	(effectful function)
ε	$::=$	(r, k, n)	(effect grade: risk × caps × resource count)
r	$::=$	ReadOnly | SafeChange | Disruptive	(risk level)
κ	$::=$	$\{c_1, \ldots, c_n\}$	(capability set)
c	$::=$	read_logs | read_metrics | read_traces	(read caps)
	|	kubectl_restart | kubectl_scale	(mutation caps)
	|	rollback_release | config_update | $\cdots$	
e	$::=$	$x \mid \lambda x{:}\tau.\ e \mid e_1\ e_2$	(core λ-calculus)
	|	**let** $x = e_1$ **in** e_2	(binding)
	|	$\textbf{observe}_c\ e$	(diagnosis query)
	|	$\textbf{act}_c\ e$	(mitigation action)
	|	**guard** $e_{cond}\ e_{body}$	(conditional guard)
	|	**branch** $e\ \overline{p_i \Rightarrow e_i}$	(pattern branch)
	|	**retry** $n\ e$	(bounded retry)
	|	**rollback** e_{token}	(rollback)
	|	**require_approval** e	(approval gate)
	|	**hypothesize** $e_{evidence}$	(form hypothesis)
	|	**validate** $e_{hyp}\ e_{check}$	(validate hypothesis)
	|	$\Lambda\alpha \sqsubseteq \varepsilon.\ e$	(effect abstraction)
	|	$e\ [\varepsilon]$	(effect application)

Fig. 3. Core monomorphic syntax of RunbookFX, with domain-specific constructs for incident diagnosis (observe, hypothesize, validate) and mitigation (act, guard, branch, retry, rollback, require_approval). Types are annotated with effect grades $\varepsilon = (r, k, n)$ from the parametric effect algebra (§5.2). The $\text{RollbackToken}^{\text{aff}}$ type is affine: tokens must be used at most once (§5.4). A sketched bounded effect polymorphism extension that adds $\varepsilon ::= \cdots \mid \alpha$ and $\tau ::= \cdots \mid \forall\alpha \sqsubseteq \varepsilon_b.\ \tau$ appears in §5.6.

the prior value, for a scale-up it records the prior replica count, and for a pod restart it records the prior pod spec; **rollback** invokes the handler to restore the c-component of world state to its pre-**act** value (precise statement: Definition 6.1). The affine qualifier ensures that each token is consumed at most once (§5.4), preventing double-rollback bugs. The **guard** combinator conditions execution on a runtime predicate, such as "error rate below threshold", to prevent blind execution. The **branch** construct enables multi-way dispatch based on diagnostic results. The **retry** combinator provides bounded retry with configurable backoff. The **require_approval** gate wraps any Disruptive action, blocking execution until explicit approval is obtained.

4.3 A Running Example

Listing 1 shows a complete RunbookFX program that diagnoses and mitigates a latency spike incident. The listing uses do-notation, pipeline (| >), and `if/then/else` as syntactic sugar over the core syntax; desugaring rules appear in the supplementary material. The program first queries traces to identify the slow service, then examines logs to confirm that connection pool exhaustion is the root cause, and finally executes a guarded restart with rollback capability only after validating the diagnosis.

Listing 1. A RunbookFX program for latency-spike mitigation. The program diagnoses connection pool exhaustion via trace and log analysis, then executes a guarded restart with rollback.

```
1  -- Diagnose: trace analysis -> log verification -> hypothesis
2  let traces = observe[read_traces] (queryTraces incident "p99 > 500ms")
3  let slowSvc = summarize traces byLatency |> head
4  let logs = observe[read_logs] (queryLogs slowSvc "connection pool")
5  let evidence = correlate traces logs
6  let hyp = hypothesize evidence "connection_pool_exhaustion"
7
8  -- Validate hypothesis before acting
9  let confirmed = validate hyp (do
10   let poolMetrics = observe[read_metrics] (queryMetrics slowSvc "pool_active")
11   return (poolMetrics.current > poolMetrics.max * 0.95))
12
13 -- Mitigate: guarded restart with rollback
14 guard confirmed (do
15   let (result, token) = act[kubectl_restart] (restartPod slowSvc)
16   -- Verify recovery
17   let recovered = observe[read_metrics] (queryMetrics slowSvc "error_rate")
18   if recovered.current < 0.01
19     then return (Success token)
20     else do rollback token
21             return (Failure "restart did not resolve"))
```

The example illustrates several key RunbookFX properties. Diagnosis precedes mitigation: lines 2–6 gather evidence before any state change. The hypothesis is validated before acting in lines 9–11. The restart action on line 15 produces an affine rollback token; lines 17–21 verify recovery with a fallback to rollback on failure.

The type system statically ensures three properties. First, the restart action requires its corresponding capability kubectl_restart $\in \kappa$. Second, the action's marginal effect of SafeChange risk with resource count 1 composes correctly with the read capabilities from diagnosis in §5.3. Third, the affine rollback token is consumed exactly once, either returned on success at line 19 or consumed by rollback on failure at line 20. We formalize these guarantees in the type system in §5.

5 Type and Effect System

The type system of RunbookFX is the central technical contribution of this paper. It combines three ideas from the programming languages literature, capability-based authorization, graded effect tracking, and affine resource management, into a unified system tailored to the safety requirements of incident response. We first present the typing judgment and the parametric effect algebra in §5.2, then detail three enforcement mechanisms in turn: capability tracking in §5.3, affine rollback resources in §5.4, and approval gates in §5.5. The metatheory follows in §5.7.

5.1 Typing Judgments

The core typing judgment has the form

$$\Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ \tau \triangleright \varepsilon$$

where Γ_u is the *unrestricted* type environment, Γ_a is the *affine* environment of variables that must be used at most once, κ is the granted capability set, τ is the result type, and ε is an *effect grade* drawn from a parametric effect algebra that tracks the cumulative side-effect profile.

The split into unrestricted and affine contexts follows the practical affine type systems of Tov and Pucella [2011] and the recent Affect system of van Rooij and Krebbers [2025]. In RunbookFX, the affine context tracks *rollback tokens*: every state-changing action produces a token that must be either consumed by **rollback**, returned to the caller, or discarded with a static warning. This provides a principled guarantee that rollback capability is preserved throughout execution, replacing the ad-hoc must-use checks common in infrastructure tooling.

5.2 Parametric Effect Algebra

Rather than hardcoding a fixed set of effect labels, RunbookFX parameterizes its type system over an *effect algebra* in the sense of Yoshioka et al. [2024]. This generalization provides a clean algebraic foundation and enables instantiation to different operational domains, such as compliance and multi-tenant isolation, without modifying the core type theory.

Definition 5.1 (Effect Monoid). An effect monoid is a preordered commutative monoid $\mathcal{E} = (E, \bot_{\mathcal{E}}, \sqcup, \sqsubseteq)$ satisfying identity ($\bot_{\mathcal{E}} \sqcup \varepsilon = \varepsilon$), commutativity, associativity, and monotonicity: if $\varepsilon_1 \sqsubseteq \varepsilon_1'$ and $\varepsilon_2 \sqsubseteq \varepsilon_2'$, then $\varepsilon_1 \sqcup \varepsilon_2 \sqsubseteq \varepsilon_1' \sqcup \varepsilon_2'$.

This generalizes the bounded join-semilattice formulation common in effect systems [Yoshioka et al. 2024] by dropping idempotence, enabling *additive* resource tracking alongside the standard lattice-based risk and capability components. The RunbookFX effect algebra is a *product* of three domain-specific components:

$$\mathcal{E}_{\mathsf{RunbookFX}} = \mathsf{Risk} \times \mathcal{K} \times \mathbb{N}.$$

Risk component Risk. The three-element lattice ReadOnly $\sqsubseteq$ SafeChange $\sqsubseteq$ Disruptive classifies operations by mutation severity. ReadOnly operations such as telemetry queries carry no mutation risk. SafeChange operations such as pod restarts and configuration updates modify state reversibly. Disruptive operations such as data deletion and major rollbacks carry significant risk and require explicit approval.

Capability-usage component $\mathcal{K}$. The power set $(\mathcal{P}(\mathsf{Cap}), \emptyset, \cup, \subseteq)$ tracks which capabilities have been *exercised* during computation. This is distinct from the granted set κ: the grant κ is a static upper bound on what the program *may* use, while the effect component $k \subseteq \kappa$ records what was *actually* used. The distinction enables precise audit: the effect annotation reveals exactly which infrastructure tools a runbook invoked, without inspecting the execution trace.

Resource component $\mathbb{N}$. The natural numbers $(\mathbb{N}, 0, +, \leq)$ track the cumulative number of rollback obligations produced during execution. This component interacts with the affine context: each **act** operation contributes 1 to the resource count, reflecting a new rollback obligation. Unlike the risk and capability components, the resource component is *additive* rather than idempotent: two sequential actions produce a combined count of 2. The additive monoid $(\mathbb{N}, 0, +)$ is not a semilattice, motivating our generalized Definition 5.1.

The product effect is ordered componentwise:

$$(r_1, k_1, n_1) \sqsubseteq (r_2, k_2, n_2) \iff r_1 \sqsubseteq r_2 \ \wedge \ k_1 \subseteq k_2 \ \wedge \ n_1 \leq n_2,$$

with $\sqcup$ defined componentwise: risk takes the lattice join, capabilities take the union, resources take the sum. Sequential composition combines effects via $\sqcup$:

$$\varepsilon(\textbf{let } x = e_1 \textbf{ in } e_2) = \varepsilon(e_1) \sqcup \varepsilon(e_2).$$

This product construction satisfies the safety conditions of Yoshioka et al. [2024]. Although their framework assumes a semilattice, the conditions themselves only require a preordered monoid; the proofs of their abstract preservation and progress theorems go through without idempotence, since the key inductive invariants hold for any ordered monoid. We verify the safety conditions in the supplementary material.

5.2.1 Interaction Axioms. A crucial design question is whether $\mathcal{E}_{\mathsf{RunbookFX}} = \mathsf{Risk} \times \mathcal{K} \times \mathbb{N}$ is merely a trivial product of three independent monoids. It is not: the three components interact through four *well-formedness axioms* that cross-cut the product structure, reflecting essential domain invariants.

Definition 5.2 (Well-formed Effect Grade). An effect grade $\varepsilon = (r, k, n)$ is *well-formed*, written $\mathit{wf}(\varepsilon)$, if it satisfies:

(I1) $r = \mathsf{ReadOnly} \implies n = 0$ (read operations incur no rollback obligations)
(I2) $r = \mathsf{Disruptive} \implies k \neq \emptyset$ (disruptive risk requires a capability witness)
(I3) $n > 0 \implies r \sqsupseteq \mathsf{SafeChange}$ (resource obligations entail mutation risk)
(I4) $\forall c \in k.\ \mathit{risk}(c) \sqsubseteq r$ (aggregate risk bounds each capability's risk)

Coupling risk and resources. Axioms I1 and I3 link the risk and resource components. They are logically equivalent over the three-element risk lattice, since $r \sqsupseteq \mathsf{SafeChange}$ is equivalent to $r \neq \mathsf{ReadOnly}$ and I3 is the contrapositive of I1. We retain both for clarity: I1 is the natural invariant from the read-only perspective, while I3 captures the resource-accounting view. Note that together they do not form a full biconditional, since a SafeChange computation may have $n = 0$ after effect subsumption; the axioms only exclude the pathological case of ReadOnly with outstanding obligations.

Capability witnesses and aggregate bounds. Axiom I2 prevents *phantom risk*: without it, the grade $(\mathsf{Disruptive}, \emptyset, 0)$ would be well-formed, admitting a computation classified as disruptive yet exercising no capability, which would make the capability-usage component unreliable for audit. Axiom I4 ensures the risk component is a sound upper bound on the aggregate risk of all exercised capabilities; it is a closure condition derivable from the monoid structure, but we include it to make the well-formedness invariant self-contained.

Necessity for safety. Each axiom family is *necessary* for a corresponding safety theorem. The I1/I3 pair excludes the grade $(\mathsf{ReadOnly}, \emptyset, 1)$, a read-only computation carrying an outstanding rollback obligation with no mutation to roll back, which would otherwise violate Theorem 5.6. Dropping I2 admits $(\mathsf{Disruptive}, \emptyset, 0)$ despite no capability use; dropping I4 admits a grade (r, k, n) with $r < \bigsqcup_{c \in k} \mathit{risk}(c)$ that misrepresents actual risk. The axioms are also essential for soundness of effect subsumption in T-Sub: without the $\mathit{wf}(\varepsilon')$ premise, subsumption could escalate risk while creating inconsistent grades. The technical contribution is not the axioms individually but their *composition-preservation property* of Theorem 5.3: the effect join of two well-formed grades is again well-formed, despite operating on a non-idempotent additive component, enabling the safety theorems to propagate through sequential composition, branching, and effect subsumption.

Theorem 5.3 (Effect Well-formedness Preservation). *If* $\Gamma_u;\ \Gamma_a;\ \kappa \vdash e : \tau \triangleright \varepsilon$, *then* $\mathit{wf}(\varepsilon)$. *Moreover, if* $\mathit{wf}(\varepsilon_1)$ *and* $\mathit{wf}(\varepsilon_2)$, *then* $\mathit{wf}(\varepsilon_1 \sqcup \varepsilon_2)$.

Proof sketch. Base cases follow by inspection: T-Observe produces $(\mathsf{ReadOnly}, \{c\}, 0)$ and T-Act produces $(\mathit{risk}(c), \{c\}, 1)$, both satisfying (I1)–(I4). For composition $\varepsilon_1 \sqcup \varepsilon_2 = (r_1 \sqcup r_2, k_1 \cup k_2, n_1 + n_2)$, each axiom is preserved because the risk join, capability union, and resource sum respect the cross-cutting constraints. Full details in the supplementary material. □ □

The interaction axioms identify the product algebra as a *constrained product* rather than a flat one: components are coupled through domain-specific invariants that composition preserves. This provides additional reasoning power; for example, I3 flags computations claiming ReadOnly risk with nonzero rollback obligations as internally inconsistent, ruling them out of consideration entirely.

5.3 Capability Tracking

Each tool invocation $\mathbf{observe}_c$ or $\mathbf{act}_c$ requires that capability c is present in the granted set κ. The typing rules enforce this statically:

$$\textsc{T-Observe}\quad \frac{c \in \kappa \qquad \Gamma;\ \kappa \vdash e\ :\ \mathsf{Query}_c \triangleright \varepsilon'}{\Gamma;\ \kappa \vdash \mathbf{observe}_c\ e\ :\ \mathsf{Result}_c \triangleright \varepsilon' \sqcup (\mathsf{ReadOnly}, \{c\}, 0)}$$

$$\textsc{T-Act}\quad \frac{c \in \kappa \qquad \Gamma;\ \kappa \vdash e\ :\ \mathsf{Action}_c \triangleright \varepsilon'}{\Gamma;\ \kappa \vdash \mathbf{act}_c\ e\ :\ \tau_c \times \mathsf{RollbackToken}^{\mathsf{aff}} \triangleright \varepsilon' \sqcup (\mathit{risk}(c), \{c\}, 1)}$$

where Γ abbreviates $\Gamma_u;\ \Gamma_a$. The function $\mathit{risk} : \mathsf{Cap} \to \mathsf{Risk}$ maps each capability to its minimum risk level: read capabilities map to ReadOnly, mutation capabilities to SafeChange or Disruptive depending on destructive potential. T-Act returns a pair $\tau_c \times \mathsf{RollbackToken}^{\mathsf{aff}}$ whose second component carries an affine qualifier; the token enters the affine context Γ_a when the pair is destructured via pattern binding (rule T-Let-Pair, Appendix A). The two-step design separates effect production from resource introduction, following standard practice in affine type systems [Tov and Pucella 2011].

The full effect derivation for Listing 1 appears in Appendix C. Diagnosis contributes a ReadOnly effect with two capabilities and zero rollback obligations; validation adds a third capability while remaining ReadOnly; the single **act** invocation escalates risk to SafeChange and introduces the first rollback obligation. The full program effect satisfies all four interaction axioms.

Design choice: input κ vs. synthesized k'. An alternative design carries no granted set in the judgment and merely synthesizes the used set $k' \subseteq \mathsf{Cap}$ as part of the effect, deferring the authorization check $k' \subseteq \kappa$ to a single top-level test. Both designs accept the same set of programs (the equivalence is straightforward: any derivation in the input-κ system maps to one in the synthesized-k' system by erasing κ from premises, and conversely the top-level test makes the inclusion explicit). We chose the input-κ formulation for two pragmatic reasons. First, it produces *localized* capability errors at the offending $\mathbf{act}_c$ or $\mathbf{observe}_c$ site, which the CEGIS repair loop (§7) feeds back to the LLM as a single targeted hint. A top-level pass collects all violations at once, which is also useful, but in our pipeline the LLM tends to respond better to a focused single-site hint than to an exhaustive set; reporting both is straightforward, and a deployment that prefers batched feedback can switch to the synthesized-k' presentation without changing the metatheory. Second, threading κ through the rules makes capability-scoped subderivations (e.g., a polymorphic combinator quantified over an effect bound ε_b whose capability component is restricted) directly expressible without an auxiliary scoping mechanism. Either presentation supports the same metatheory; implementations preferring batched feedback can switch to the synthesized-k' form without changing any theorem. In our pipeline, focused single-site hints converged faster than aggregated reports under GPT-4o.

The trade-off, candidly, is one of aesthetics versus pragmatics. The synthesized-k' form is the more parsimonious declarative system: it eliminates a context from every judgment, and its single

top-level $k' \subseteq \kappa$ check is independent of typing-rule order and can be implemented with any standard effect-aggregation pass. An implementation built on top of the synthesized-k' system can still record the source location of each effect contribution and emit a multi-violation report, so the LLM-feedback argument is not exclusive to the input-κ form. On the operational side, the input-κ form makes the polymorphic-combinator scoping (§5.6) a one-line rule rather than requiring an auxiliary capability-scope abstraction, and it expresses the affine-context restriction of **retry** and **par** uniformly with the capability restriction without introducing a second well-formedness predicate. Reasonable readers can disagree on the weighting; we report the choice along with its alternatives so that downstream implementers can pick the formulation that matches their tooling, in either case without revisiting the metatheory.

5.4 Affine Rollback Resource Management

The treatment of rollback tokens as affine resources is a key innovation of RunbookFX. Every $\mathbf{act}_c$ operation produces a pair $(\tau_c, \mathsf{RollbackToken})$, where the token enters the affine context. The affine discipline ensures that each token is consumed *at most once*: it may be used for rollback, returned to the caller, or explicitly discarded with a static warning, but it cannot be duplicated or used multiple times.

Context splitting. When typing a binding $\mathbf{let}\ x = e_1\ \mathbf{in}\ e_2$, the affine context $\Gamma_a = \Gamma_{a1} \boxtimes \Gamma_{a2}$ is split disjointly between subexpressions ($\Gamma_{a1} \cap \Gamma_{a2} = \emptyset, \Gamma_{a1} \cup \Gamma_{a2} = \Gamma_a$), assigning each affine variable to exactly one subexpression. This is the standard affine context splitting from substructural type theory [Walker 2005]; the full T-Let rule appears in Appendix A.

Guard and branching. The guard construct conditionally executes a body, so both subexpressions may run and the affine context is *split* between them. The branch construct dispatches to exactly one arm at runtime, so all arms may *share* the affine context:

$$\textsc{T-Guard}\quad \frac{\Gamma_u;\ \Gamma_{a1};\ \kappa \vdash e_c\ :\ \mathsf{Bool} \triangleright \varepsilon_c \qquad \Gamma_u;\ \Gamma_{a2};\ \kappa \vdash e_b\ :\ \tau \triangleright \varepsilon_b \qquad \Gamma_a = \Gamma_{a1} \boxtimes \Gamma_{a2}}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash \mathbf{guard}\ e_c\ e_b\ :\ \mathsf{Option}\ \tau \triangleright \varepsilon_c \sqcup \varepsilon_b}$$

$$\textsc{T-Branch}\quad \frac{\Gamma_u;\ \Gamma_{a0};\ \kappa \vdash e\ :\ \tau_s \triangleright \varepsilon_0 \qquad \forall i.\ \Gamma_u, x_i : \tau_i;\ \Gamma_{a'};\ \kappa \vdash e_i\ :\ \tau \triangleright \varepsilon_i \qquad \Gamma_a = \Gamma_{a0} \boxtimes \Gamma_{a'}}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash \mathbf{branch}\ e\ \overline{p_i \Rightarrow e_i}\ :\ \tau \triangleright \varepsilon_0 \sqcup \bigsqcup_i \varepsilon_i}$$

In T-Branch, the overall effect $\varepsilon_0 \sqcup \bigsqcup_i \varepsilon_i$ combines two distinct operations: $\sqcup$ is the monoid operation under which resources *sum*, while $\bigsqcup_i$ is the lattice join over branch effects under which resources take the *maximum*, since at most one branch executes at runtime. The affine context $\Gamma_{a'}$ is shared across all branches because mutual exclusivity, enforced by deterministic pattern matching in E-Branch, guarantees each token is consumed at most once.

The choice of affine over linear types is deliberate. Linear types would require exactly-once consumption, forcing every branch to consume every shared token, an impractical restriction when some branches represent "no action needed" paths. T-Guard's false branch discards the body's affine context, which would be ill-typed under a linear discipline.

Rollback consumption. The rollback operation consumes a token from the affine context:

$$\textsc{T-Rollback}\quad \frac{\Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ \mathsf{RollbackToken} \triangleright \varepsilon'}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash \mathbf{rollback}\ e\ :\ \mathsf{Unit} \triangleright \varepsilon' \sqcup (\mathsf{SafeChange}, \emptyset, 0)}$$

When e is an affine variable t, the variable rule (T-Var-Aff, Appendix A) removes t from Γ_a, preventing subsequent use. The companion weakening rule T-Weaken-Aff allows discarding an unused token but emits a static warning. The synthesis pipeline treats this warning as a type error, tightening the affine discipline to an "effectively linear" guarantee for synthesized code while retaining the simpler affine formalization. T-Abs requires an empty affine context, preventing lambda abstractions from capturing rollback tokens; this restriction is standard for affine λ-calculi [Tov and Pucella 2011] and does not limit expressivity, since synthesized runbooks are first-order.

Omitted for space, but the affine discipline applies with well-typed and ill-typed programs. In Listing 1, the success branch returns the token as part of the result, removing it from Γ_a, while the failure branch consumes it via **rollback**. Since T-Guard splits the affine context and the two branches are mutually exclusive, the token is consumed exactly once on either execution path.

5.5 Approval Gates

Disruptive operations require explicit approval. The typing rule below ensures any expression whose effect includes Disruptive risk is wrapped in **require_approval**:

$$\textsc{T-Approve}\quad \frac{\Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ \tau \triangleright (r,k,n) \qquad r = \mathsf{Disruptive}}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash \textbf{require_approval}\ e\ :\ \tau \triangleright (\mathsf{Disruptive},k,n)}$$

The type checker further enforces a *disruptive-action check* as a top-level well-formedness condition, analogous to Koka's main-effect restriction: a runbook program $\emptyset;\ \emptyset;\ \kappa \vdash p\ :\ \tau \triangleright (r,k,n)$ with $r = \mathsf{Disruptive}$ is accepted only if p has the syntactic form **require_approval** e. This is a syntactic check on the program's outermost constructor; the typing rules allow bare Disruptive expressions at intermediate positions enclosed by an approval gate. If the top-level check fails, the type checker emits a structured error that drives the repair loop (§7) to add the missing gate.

The remaining typing rules (T-Retry with empty affine context, T-Hypothesize, T-Validate, and effect subsumption T-Sub) follow the same design principles; their full statements appear in Appendix A.

5.6 Bounded Effect Polymorphism (Sketched Extension)

A simply-typed DSL would require a separate combinator for every effect level, with distinct $\mathit{diagnoseAndAct}_{\mathsf{SafeChange}}$ and $\mathit{diagnoseAndAct}_{\mathsf{Disruptive}}$ combinators that differ only in their effect annotation. To support reusable, generic combinators without sacrificing decidable type checking, we sketch a *bounded effect polymorphism* extension. The extended grammar adds an effect-variable production $\varepsilon ::= \cdots \mid \alpha$ and a quantified type $\tau ::= \cdots \mid \forall \alpha \sqsubseteq \varepsilon_b.\ \tau$, where bounds ε_b range over concrete (closed) effect grades. The typing judgment becomes $\Delta;\ \Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ \tau \triangleright \varepsilon$, with Δ an *effect-variable context* mapping each variable α to its upper bound. The same context Δ is threaded uniformly through every typing rule of §5.1–§5.5; in the monomorphic fragment $\Delta = \emptyset$ and the rules reduce verbatim to those already presented.

$$\textsc{T-EffAbs}\quad \frac{\Delta, \alpha \sqsubseteq \varepsilon_b;\ \Gamma_u;\ \emptyset;\ \kappa \vdash e\ :\ \tau \triangleright \varepsilon' \qquad \mathit{wf}(\varepsilon_b)}{\Delta;\ \Gamma_u;\ \emptyset;\ \kappa \vdash \Lambda\alpha \sqsubseteq \varepsilon_b.\ e\ :\ (\forall \alpha \sqsubseteq \varepsilon_b.\ \tau) \triangleright (\mathsf{ReadOnly},\emptyset,0)}$$

$$\textsc{T-EffApp}\quad \frac{\Delta;\ \Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ (\forall \alpha \sqsubseteq \varepsilon_b.\ \tau) \triangleright \varepsilon' \qquad \varepsilon_{arg} \sqsubseteq \varepsilon_b \qquad \mathit{wf}(\varepsilon_{arg})}{\Delta;\ \Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ [\varepsilon_{arg}]\ :\ \tau[\varepsilon_{arg}/\alpha] \triangleright \varepsilon'}$$

Three design decisions balance expressiveness and tractability. T-EffAbs requires an empty affine context, preventing token aliasing across instantiations, and the new $wf(\varepsilon_b)$ premise rules out abstractions over inconsistent bounds. Bounds ε_b are concrete grades rather than variables, ensuring bound checking $\varepsilon_{arg} \sqsubseteq \varepsilon_b$ is decidable via componentwise comparison on a finite lattice, set inclusion, and natural number comparison. Effect variables quantify only over effects and not over types, keeping the polymorphism lightweight. A polymorphic combinator can abstract over the effect level of the mitigation action while sharing diagnostic logic; the LLM synthesizes monomorphic programs, and the combinator library provides polymorphic building blocks.

Status. The polymorphic extension is presented as a standard but currently *unmechanized* extension of the monomorphic core. Theorem 5.8 (Polymorphic Combinator Safety, §5.7) is stated as a conjecture: its informal proof reuses effect substitution and the monomorphic metatheory, but neither the substitution lemma nor the case analysis on Δ is mechanized in our current Coq development (§D). All other safety theorems of §5.7 are stated and proven for the monomorphic system. We mark this distinction explicitly so that the formal guarantees in the experimental evaluation rest on the mechanized monomorphic core, while the polymorphic surface used in the combinator library is sketched.

5.7 Metatheory

The theorem statements in this subsection use the small-step reduction relation $\langle e, \sigma, \theta\rangle \xrightarrow{H} \langle e', \sigma', \theta'\rangle$ and the well-formed handler condition (Definition 6.1), both formally defined in §6.2–§6.4. A reader encountering them for the first time may skim those definitions before continuing, or accept $\xrightarrow{H}$ as a standard call-by-value reduction parameterized by a handler environment H; the safety arguments below depend only on its structural shape, not on concrete handler return values (made precise in §6.4).

We establish type safety together with three *domain-specific* safety theorems—no unauthorized action (Theorem 5.5), rollback availability (Theorem 5.6), and rollback linearity (Theorem 5.7)—in addition to the effect well-formedness preservation theorem (Theorem 5.3) already established in §5.2.1. The monomorphic core calculus, consisting of 12 typing rules from §5.1–§5.5 together with 9 reduction rules from §6.2, is mechanized in Coq at ∼2,200 lines; see Appendix D. Four theorems carry the safety guarantees of §9: Effect WF Preservation (Theorem 5.3), Progress (part 2 of Theorem 5.4), single-step No Unauthorized Action (Theorem 5.5), and Rollback Linearity (Theorem 5.7). All four close with `Qed`. Each applies whenever the verifier accepts a candidate, so the zero-violation and 100% rollback-coverage claims hold for every accepted program. Of the 27 supporting obligations in the substitution and multi-step layers, 18 close with `Qed` after the latest mechanization pass—all Canonical Forms, all effect-operation Inversion lemmas, Value Typing, de Bruijn weakening (via a generalised `weakening_u_at`), and the typing-respecting reduction cases for `observe`, `act`, `rollback`, and the affirmative guard. The remaining 9 thread through one substitution lemma whose proof follows the standard Pierce SF skeleton (cf. Pierce et al. [2019]); Section D ledgers them, and full paper proofs appear in the supplementary material.

Theorem 5.4 (Type Safety). *If* $\Gamma_u;\ \Gamma_a;\ \kappa \vdash e : \tau \triangleright \varepsilon$, *then the following three properties hold:* ***(part 1: Preservation)*** *If* $\langle e, \sigma, \theta\rangle \longrightarrow \langle e', \sigma', \theta'\rangle$, *then* $\Gamma'_u;\ \Gamma'_a;\ \kappa \vdash e' : \tau \triangleright \varepsilon'$ *with* $\Gamma'_u \supseteq \Gamma_u, \Gamma'_a \subseteq \Gamma_a$, *and* $\varepsilon' \sqsubseteq \varepsilon$: *effects cannot grow and affine tokens cannot be duplicated.* ***(part 2: Progress)*** *If* e *is not a value and the handler environment* H *is well-formed w.r.t.* κ *(Definition 6.1), then* $\langle e, \sigma, \theta\rangle$ *can step.* ***(part 3: Soundness, corollary)*** *If additionally* $\Gamma_u = \Gamma_a = \emptyset$, *evaluation either produces a value* v *with effect* $(\mathsf{ReadOnly}, \emptyset, 0)$ *or diverges;* e *never reaches a stuck state.*

Proof sketch. Preservation is by induction on reduction; progress by induction on typing, where handler totality guarantees effect operations do not get stuck; soundness is immediate. □ □

Theorem 5.5 (No Unauthorized Action). *If* $\Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ \tau \triangleright \varepsilon$ *and* $\langle e, \sigma, \theta\rangle$ *reduces to* $\langle E[\mathbf{act}_c\ v], \sigma', \theta'\rangle$*, then* $c \in \kappa$*.*

Proof sketch. Preservation ensures every reduct is well-typed. T-Act requires $c \in \kappa$, and κ is invariant under reduction. □

This theorem relates effect typing to the operational semantics in the form most relevant to incident response: any capability that ever fires in any reachable configuration is bounded by κ, the static grant. The shape of the statement reflects a deliberate semantic choice. Rather than making the operational semantics get stuck when an out-of-κ capability is encountered—which would entangle the handler signature with the typing context and require every handler implementation to inspect κ—we keep the handlers *total* over their declared interface and prove the authorization property at the type level. The same evidence ($c \in \kappa$ at the typing-rule premise) that grants progress under a well-formed handler also rules out the unauthorized-reduction configuration entirely: any candidate program that could expose such a configuration is rejected statically at type-check time. The audit trace θ in §6.5 provides the operational read-out of this connection by recording every capability witness alongside its operation, so the effect-component $k \subseteq \kappa$ tracked by typing is the run-time-observable invariant on θ.

Theorem 5.6 (Rollback Availability). *If* $\Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ \tau \triangleright (r, k, n)$ *with* $r \sqsupseteq$ SafeChange*, then for every execution trace, every token produced by* $\mathbf{act}_c$ *is either passed to a* **rollback** *expression, returned in the result, or explicitly discarded with a static warning at type-checking time.*

Proof sketch. By the affine discipline on Γ_a. T-Act places the token in Γ_a; context splitting (T-Let) assigns it to exactly one branch; T-Rollback, the return path, or T-Weaken-Aff (with warning) consume it. □

Theorem 5.7 (Rollback Linearity). *If* $\Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ :\ \tau \triangleright \varepsilon$ *and during execution of* e*,* $\mathbf{act}_c$ *produces token* t*, then* t *is consumed by* **rollback** *at most once.*

Proof sketch. Context splitting assigns t to exactly one branch; T-Rollback removes t from Γ_a, so any subsequent use fails statically. □

Theorem 5.8 (Polymorphic Combinator Safety). *If* $\Delta, \alpha \sqsubseteq \varepsilon_b;\ \Gamma_u;\ \emptyset;\ \kappa \vdash e\ :\ \tau \triangleright \varepsilon$ *and* $\varepsilon_{arg} \sqsubseteq \varepsilon_b$ *with* $\mathit{wf}(\varepsilon_{arg})$*, then* $e[\varepsilon_{arg}/\alpha]$ *satisfies Theorems 5.5–5.7.*

Proof sketch. Effect substitution preserves typing (the supplementary material) and well-formedness (the supplementary material); safety follows from the monomorphic metatheory. □

Connection to abstract safety. The monomorphic theorems—Theorem 5.4 together with Theorems 5.5–5.7—instantiate the framework of Yoshioka et al. [2024]: our product algebra satisfies their safety conditions without requiring idempotence. The three domain-specific theorems exploit the specific product structure and cannot be derived from the abstract framework alone. Theorem 5.8 (Polymorphic Combinator Safety) is a conjectured lift of these properties through effect substitution; its formal status is discussed at the end of §5.6.

6 Semantics and Runtime

We give RunbookFX an operational semantics grounded in algebraic effect handlers [Plotkin and Pretnar 2009]. The key idea is that the DSL's domain-specific operations – telemetry queries, infrastructure mutations, rollback, and approval – form an *effect signature*, and different operational modes such as live execution, replay testing, and mocking correspond to different *handlers* that interpret this signature. This decomposition provides a clean separation of concerns and enables a handler parametricity result: the safety properties established by the type system hold *regardless* of which handler is used.

6.1 Effect Signature

We define the RunbookFX effect signature Σ as a set of operation symbols with input and output types:

$$\begin{aligned}\Sigma_{\mathsf{RunbookFX}} \quad = \quad & \{\mathbf{observe}_c \,:\, \mathsf{Query}_c \to \mathsf{Result}_c \mid c \in \mathsf{Cap}_{read}\} \\ \cup \quad & \{\mathbf{act}_c \,:\, \mathsf{Action}_c \to \tau_c \times \mathsf{RollbackToken} \mid c \in \mathsf{Cap}_{mut}\} \\ \cup \quad & \{\mathbf{rollback} \,:\, \mathsf{RollbackToken} \to \mathsf{Unit}\} \\ \cup \quad & \{\mathbf{approve} \,:\, \tau \to \tau\}\end{aligned}$$

where Cap_{read} and Cap_{mut} partition the capability set into read and mutation capabilities. Each operation in Σ corresponds to a syntactic construct in the DSL. This signature is standard in the algebraic effects literature: operations are typed function symbols that, when encountered during evaluation, are delegated to an enclosing handler.

Handler typing. A handler environment H assigns interpretations to the operations in Σ as a family of typed functions: $H_c^{obs} : \mathsf{Query}_c \times \mathsf{World} \to \mathsf{Result}_c$ leaves world state unchanged; $H_c^{act} : \mathsf{Action}_c \times \mathsf{World} \to \tau_c \times \mathsf{World} \times \mathsf{RollbackToken}$ produces a modified world and a rollback token; $H^{undo} : \mathsf{RollbackToken} \times \mathsf{World} \to \mathsf{World}$ provides partial undo; and $H^{approve}$ acts as identity on approved values. The well-formedness conditions of Definition 6.1 ensure the type system's safety guarantees transfer across handlers.

6.2 Operational Semantics

A runtime configuration is a triple $\langle e, \sigma, \theta \rangle$ where e is the current expression, σ is the external world state (telemetry stores, infrastructure state), and θ is the accumulated audit trace. The small-step reduction relation takes the form:

$$\langle e, \sigma, \theta \rangle \xrightarrow{H} \langle e', \sigma', \theta' \rangle$$

where H is the active *handler environment* that gives interpretations to the effect operations. The handler annotation makes explicit that the same program may reduce differently under different handlers.

Values. We define the set of values v (fully reduced expressions):

$$v \;::=\; \lambda x{:}\tau.\, e \mid () \mid \mathsf{true} \mid \mathsf{false} \mid s \mid (v_1, v_2) \mid \mathsf{Some}\ v \mid \mathsf{None} \mid t_{tok}$$

where t_{tok} ranges over rollback token values produced by **act** operations.

Evaluation contexts. We define standard left-to-right call-by-value evaluation contexts $\mathcal{E}$ (the supplementary material).

Standard reductions. The pure, handler-independent reductions E-Beta, E-Let, E-Let-Pair, E-Guard, E-EffApp, E-Branch, E-Retry, and E-Context follow standard call-by-value evaluation; full rules appear in the supplementary material. Three design points merit attention. E-Branch uses *deterministic* pattern matching, essential for soundness of sharing affine contexts across branches in T-Branch. E-Retry uses a hybrid-semantics technique with a multi-step premise: the world

state reverts on retry, which is sound because T-Retry requires an empty affine context, while the audit trace advances. Parallel composition is restricted to ReadOnly effects, ensuring confluent interleaving.

The remaining reductions are *handler-dependent*, invoking H to interpret effect operations. Observations are pure (σ unchanged); actions produce a modified σ' and a rollback token; rollback invokes the undo handler; and approval delegates to $H^{approve}$. All four rules append a trace entry to θ:

$$\langle \mathbf{observe}_c\ v, \sigma, \theta \rangle \xrightarrow{H} \langle H_c^{obs}(v, \sigma),\ \sigma,\ \theta \cdot (\mathrm{obs}, c, v, r) \rangle$$

$$\langle \mathbf{act}_c\ v, \sigma, \theta \rangle \xrightarrow{H} \langle (r, t),\ \sigma',\ \theta \cdot (\mathrm{act}, c, v, r, t, \mathit{risk}(c)) \rangle$$

$$\langle \mathbf{rollback}\ t, \sigma, \theta \rangle \xrightarrow{H} \langle (),\ H^{undo}(t, \sigma),\ \theta \cdot (\mathrm{rollback}, t) \rangle$$

$$\langle \mathbf{require_approval}\ e, \sigma, \theta \rangle \xrightarrow{H} \langle e,\ \sigma,\ \theta \cdot (\mathrm{approved}) \rangle$$

Progress holds because well-formed handlers are total (Definition 6.1).

Reduction of Listing 1. The program reduces in three phases. First, three ReadOnly observations accumulate trace entries without modifying σ_0. Second, E-Guard-True dispatches to the mitigation branch. Third, E-Act produces mutation $\sigma_0 \to \sigma_1$ with rollback token t_1, consumed by E-Rollback on failure. The resulting trace skeleton is handler-independent by Theorem 6.3; see the supplementary material for the full derivation.

6.3 Handler Definitions

We define four handler environments that share the effect signature Σ but provide different implementations. The **live handler** H_{live} (used for ITBench) binds effect operations to real infrastructure APIs: H_c^{obs} dispatches to the Kubernetes API server, Prometheus, Jaeger, or ClickHouse depending on c; H_c^{act} executes real mutations such as `kubectl rollout restart`, with rollback tokens carrying information for `kubectl rollout undo`. The **replay handler** H_{replay} (used for RCAEval) returns pre-recorded telemetry; action operations are simulated as $H_c^{act}(v, \sigma) = (\mathit{sim}(v), \sigma, t_{noop})$ with a no-op rollback token, enabling reproducible evaluation without live infrastructure. A **mock handler** H_{mock} for unit testing and a composable **trace handler** $H_{trace}(H')$ for audit wrapping appear in the supplementary material; in production we use $H_{trace}(H_{live})$.

6.4 Handler Parametricity

The separation between the DSL semantics and handler binding yields a key compositionality result: the type system's safety guarantees are *handler-independent*.

Definition 6.1 (Well-formed Handler). A handler H is *well-formed w.r.t. capability set* κ if: (1) every H_c with $c \in \kappa$ is total; (2) every read handler H_c^{obs} is pure in σ; and (3) every mutation handler H_c^{act} produces a rollback token t such that $H^{undo}(t, \sigma')$ restores the c-component of σ. We model world state as a product $\sigma = \prod_{c \in \mathrm{Cap}} \sigma_c$ and require $\pi_c(H^{undo}(t, \sigma')) = \pi_c(\sigma)$. This componentwise condition is weaker than full world-state equality, since in practice rollback is scoped to the modified resource while other state may advance.

The three conditions ensure handlers are safe interpreters: they always terminate, read-only operations cannot hide side effects, and mutations can always be undone at the granularity of the affected resource. To formalize transfer of safety properties across handlers, we introduce a *handler bisimulation* relation capturing structural agreement.

Definition 6.2 (Handler Bisimulation). Two well-formed handlers H_1 and H_2 are *bisimilar with respect to capability set* κ, written $H_1 \sim_\kappa H_2$, if:

(B1) *Totality agreement.* For every $c \in \kappa$, H_1 defines c iff H_2 defines c, and both are total.

(B2) *Token bisimilarity.* For every $c \in \mathrm{Cap}_{mut} \cap \kappa$ and input (v, σ), if $H_{1,c}^{act}(v, \sigma) = (r_1, \sigma_1', t_1)$ and $H_{2,c}^{act}(v, \sigma) = (r_2, \sigma_2', t_2)$, then t_1 and t_2 are *token-equivalent*: $\pi_c(H_1^{undo}(t_1, \sigma_1')) = \pi_c(\sigma)$ iff $\pi_c(H_2^{undo}(t_2, \sigma_2')) = \pi_c(\sigma)$.

(B3) *Effect classification agreement.* For every $c \in \kappa$, $risk_{H_1}(c) = risk_{H_2}(c)$ (handlers agree on which operations are read-only, safe-change, or disruptive).

The bisimulation is deliberately *value-agnostic*: condition B2 does not require $r_1 = r_2$ nor $\sigma_1' = \sigma_2'$, so return values and world modifications may differ freely across handlers. Safety properties depend on which operations are invoked and how tokens flow, not on the values computed. The relation $\sim_\kappa$ is an equivalence: reflexivity and symmetry are immediate, transitivity follows because B1 and B3 are equality conditions while B2's biconditional on rollback scope composes transitively.

Theorem 6.3 (Strong Handler Parametricity). *If* $\Gamma_u;\ \Gamma_a;\ \kappa \vdash e : \tau \triangleright \varepsilon$ *and* $H_1 \sim_\kappa H_2$*, then:*

(1) *The safety properties (Theorems 5.5–5.7: no-unauthorized-action, rollback-availability, rollback-linearity) hold under* H_1 *if and only if under* H_2*.*

(2) (Conditional trace equivalence.) *When executing* e *under* H_1 *and* H_2 *follows the same control-flow path (i.e., all branch conditions evaluate identically), the resulting audit traces* θ_1 *and* θ_2 *are* structurally equivalent*: they contain the same sequence of operation kinds, capability identifiers, and risk classifications, though the concrete values and world states may differ. When different handlers induce different branch outcomes, the traces may diverge structurally; however, every possible trace still satisfies the safety properties of property (1).*

Proof sketch. The proof has two layers. *Structural:* no-unauthorized-action and rollback linearity depend only on the typing derivation, namely $c \in \kappa$ in T-Act and affine splitting in T-Let, and hold under any handler satisfying totality B1. *Semantic:* rollback availability transfers via token bisimilarity B2, while effect classification agreement B3 ensures consistent well-formedness axioms. The key subtlety is E-Branch, where different handlers may induce different branch outcomes; T-Branch types each branch under the same affine context $\Gamma_{a'}$ and the join $\bigsqcup_i \varepsilon_i$ overapproximates every branch effect, so safety holds regardless of which branch the handler selects. Full proof in the supplementary material. □ □

Practical significance. The theorem formalizes the "verify offline, execute live" workflow: a runbook verified under H_{replay} is guaranteed safe under H_{live} once $H_{replay} \sim_\kappa H_{live}$. Value-agnostic bisimulation is essential here: replay handlers return simulated telemetry while live handlers return production data, so standard handler equivalence requiring return-value agreement would incorrectly reject the pair. Our four handlers are pairwise bisimilar; the supplementary material gives the verification details. Verifying a new handler reduces to three checks at registration time, namely interface coverage for B1, token-equivalence against replay on representative inputs for B2, and risk-table agreement for B3.

6.5 Audit Trace and Graded Monadic Interpretation

The trace θ accumulated during execution is a sequence of entries $\langle op, cap, in, out, risk, time \rangle$ that the operational semantics appends to θ at every **observe**, **act**, **rollback**, or **require_approval** step. This single trace artifact supports three downstream consumers external to the language. *Compliance audit:* an external auditor (or an SOC 2/ISO 27001 compliance tool) replays θ to confirm that every recorded **act** carries an in-band capability witness; entries are append-only and

produced uniformly by the semantics, so an honest handler cannot omit an operation without violating progress. *Evidence grounding:* when a synthesized runbook concludes "the root cause is connection-pool exhaustion", θ exposes the specific $\mathbf{observe}_{read_metrics}$ and $\mathbf{observe}_{read_logs}$ entries that produced the values feeding the hypothesis, so a human operator can re-derive the diagnosis from raw telemetry rather than trust the LLM's narrative; the type system already guarantees that **hypothesize** and **validate** only consume values flowing from prior **observe** entries (§5.3), and the trace exhibits those entries explicitly. *Replay-based testing:* given a recorded θ from one handler, an alternative bisimilar handler reproduces the structural trace by Theorem 6.3, which is the mechanism behind the offline test harness of §8.

The effect tracking also admits a graded monadic interpretation [Gaboardi et al. 2016; Orchard et al. 2020] where **return** has grade $\varepsilon_{\perp}$ and **bind** composes grades via $\sqcup$; the graded monad laws follow directly from the effect monoid laws of Definition 5.1, and T-Let is precisely the graded bind rule specialized to our algebra. the supplementary material gives the full verification.

7 LLM-Guided Synthesis with Verification-as-Ranking

This section describes how RunbookFX uses LLMs as *constrained program synthesizers* rather than direct action executors, combining the generative power of language models with the safety guarantees of the type system.

7.1 Synthesis Representation

The LLM receives a structured prompt containing the RunbookFX grammar specification from Figure 3, the incident context comprising alert payload, telemetry snippets, and topology, the granted capability set κ with maximum risk level $\varepsilon_{\max}$, and a library of reusable combinator examples from historical runbooks. The LLM generates *monomorphic* candidate programs as structured DSL ASTs encoded in JSON, parsed into the RunbookFX internal representation; we use constrained decoding to ensure syntactic well-formedness, rejecting malformed outputs at generation time.

The combinator library provides *polymorphic* building blocks using bounded effect polymorphism (§5.6), which the LLM instantiates at specific effect levels. This separation keeps the synthesis task tractable while enabling reuse across incidents with different risk profiles.

Runtime contracts. Beyond the static type system, the synthesis specification carries a finite set of *runtime contracts* $\Phi = \{\phi_1, \ldots, \phi_m\}$, each a quantifier-free first-order predicate over the audit trace θ (§6.5). Contracts encode behavioral properties that the type grammar cannot statically express: e.g., "every $\mathbf{act}_c$ on a service is preceded by an **observe** on the same service" (causality), "no two **act** entries within a 5-minute window touch overlapping resources" (rate limiting), or "post-action error rate is no worse than pre-action" (recovery check). A *contract violation* during replay is a ϕ_j that evaluates to false on θ; a *replay failure* is a runtime error in the replay handler itself, typically arising when a **guard** predicate references a telemetry field absent from the offline dataset. Both are distinct from *type errors*, which are caught statically before any handler is invoked. The default contract set Φ shipped with RunbookFX is listed in the supplementary material; users may extend it per deployment.

7.2 Multi-Candidate Generation and Ranking

For each incident, the LLM generates $k = 5$ candidate runbook programs at temperature $T = 0.2$. Candidates are ranked by a multi-criteria scoring function:

$$score(p) = w_{tc} \cdot \mathbb{1}[\text{type-checks}] + w_{cv} \cdot contract_score(p) + w_{rp} \cdot replay_score(p) - w_{cost} \cdot cost(p),$$

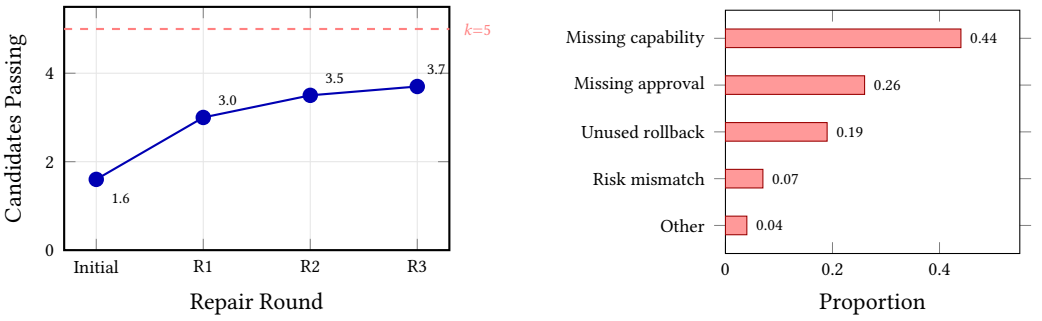
where $\mathbb{1}$[type-checks] flags static verification, *contract_score* measures the fraction of runtime contracts satisfied during replay, *replay_score* measures diagnostic accuracy against ground-truth labels when available, and *cost* penalizes excessive steps or high-risk operations. We set $w_{tc} = 10$, $w_{cv} = 5$, $w_{rp} = 3$, $w_{cost} = 1$; the dominant $w_{tc} = 10$ ensures no ill-typed candidate outscores a well-typed one, since the maximum non-safety score $w_{cv} + w_{rp} = 8 < 10$. A sensitivity analysis in Appendix F confirms robustness to ±30% perturbations of w_{cv}, w_{rp}, w_{cost} provided w_{tc} remains dominant. Verification thus acts as both a filter and a ranking mechanism: type checking is a hard constraint, while contract and replay scores rank among the verified candidates, ensuring safety is never traded for perceived effectiveness.

7.3 Counterexample-Guided Repair (CEGIS)

When all k candidates fail verification, RunbookFX enters a three-step repair loop inspired by CEGIS [Solar-Lezama et al. 2006]. *Step 1, Failure Classification:* each failing candidate is annotated with a structured failure report covering type errors, capability violations, contract violations, and replay failures, each carrying source locations and expected-vs.-actual data. For example, a candidate that issues $\mathbf{act}_{kubectl_restart}$ on `prod-api` without a preceding **observe** on that service produces a contract violation against the causality predicate ϕ_{causal}; a candidate whose **guard** predicate dereferences a missing telemetry field on the offline dataset produces a replay failure carrying the offending field name. *Step 2, Hint Construction:* failure reports are transformed into natural-language repair hints; a capability violation on `act[kubectl_restart]` where `kubectl_restart` is not in the granted set produces a hint suggesting either a diagnostic-only fallback or escalation via `require_approval`. *Step 3, Targeted Re-synthesis:* the LLM receives the original program, the failure report, and the repair hint, and generates a repair instructed to modify only failing regions while preserving the diagnostic logic. The repaired candidate re-enters the verification pipeline.

This loop iterates for at most $N_{\max} = 3$ rounds. Figure 4(a) shows the convergence: the mean number of candidates passing type check rises from 1.6 to 3.7 out of k=5 over three rounds, with the first round contributing the largest gain. Figure 4(b) shows the initial type-error distribution, dominated by missing capabilities at 44% and missing approval gates at 26%. The repair loop resolves 67% of type errors and 39% of contract violations in the first round, with marginal resolution rates dropping to 31%/23% in round 2 and 12%/8% in round 3, motivating the fixed bound (full breakdown in the supplementary material). The repair loop is a *heuristic* that guides the LLM toward well-typed programs and does not guarantee convergence; the formal guarantee of Theorem 7.2 applies only to candidates that *pass* verification.

Walkthrough. A representative invocation makes the round-by-round behavior concrete. The alert is the latency spike of Listing 1; the granted set κ permits `read_traces`, `read_logs`, `read_metrics`, and `config_update`. Round 0 produces five candidates: two are well-typed but call `act[kubectl_restart]` (a capability outside κ), one omits the **validate** between **hypothesize** and **act** (violating the causality contract ϕ_{causal}), one returns the rollback token in only one of the two **guard** arms (failing affine linearity), and one passes. Step 1 emits four reports; the failure summarizer turns the capability-violation reports into the hint "`kubectl_restart` is not granted; substitute `config_update` on the database proxy or escalate via **require_approval**", the causality report into "insert a **validate** on the hypothesis before **act**", and the linearity report into "thread `token` through both branches of the post-`act` **guard**". Step 3 instructs the LLM to apply each hint to its corresponding candidate. Round 1 yields three candidates that pass type checking, all of which then satisfy the runtime contracts on replay; ranking selects the candidate with the highest contract-replay score for execution.



(a) Repair convergence (out of k=5)

(b) Initial type error distribution

Fig. 4. Synthesis pipeline analysis. (a) Mean number of candidates passing type check across repair rounds, showing diminishing returns after round 1. (b) Distribution of type errors in initial LLM-generated candidates; missing capabilities account for 44% of errors.

7.4 Formal Properties of the Synthesis Loop

We clarify the relationship to classical CEGIS [Solar-Lezama et al. 2006]. In the standard formulation, the verifier produces a concrete *input* counterexample that the synthesizer must satisfy; convergence is typically characterized by a finite hypothesis space. Here, the "counterexamples" are *structured verification failures*, namely type errors, capability violations, and contract breaches, that serve as repair guidance rather than input/output constraints. The repair itself is LLM-driven rather than algorithmic: correctness comes not from the repair process but from the post-repair verification of Theorem 7.2. We retain the term *CEGIS-style* to emphasize the shared iterative verify-refine structure, while noting that the formal guarantees reside in the verifier, not the repair loop.

To establish that the repair loop preserves the safety guarantees of §5, we formalize what constitutes a correct synthesis task and what guarantees accepted candidates inherit.

Definition 7.1 (Synthesis Specification). A synthesis specification is a triple $S = (\kappa, \varepsilon_{\max}, \Phi)$ where κ is the granted capability set, $\varepsilon_{\max}$ is the maximum permitted effect grade, and $\Phi = \{\phi_1, \ldots, \phi_m\}$ is a set of runtime contracts over execution traces.

A specification S thus describes both the authorization boundary – which capabilities are available – and the correctness boundary – which effect levels and runtime properties are acceptable. The repair loop searches for candidates satisfying both simultaneously.

Theorem 7.2 (Synthesis-to-Execution Safety). *If the repair loop terminates with a candidate program p that passes static verification under specification $S = (\kappa, \varepsilon_{\max}, \Phi)$ and dynamic verification under replay handler H_{replay}, then for* any *well-formed handler H, including H_{live}:*

(1) *p is well-typed as $\emptyset$; $\emptyset$; $\kappa \vdash p : \tau \triangleright \varepsilon$ with $\varepsilon \sqsubseteq \varepsilon_{\max}$.*
(2) *Executing p under H satisfies no-unauthorized-action, rollback-availability, and rollback-linearity (Theorems 5.5–5.7).*
(3) *For every contract $\phi \in \Phi$, the replay execution of p under H_{replay} satisfies ϕ.*

This theorem combines type safety from §5.7 with handler parametricity from §6.4. The nontrivial content is property 2: a program verified under H_{replay} is safe under *any* well-formed handler, bridging offline verification and production safety.

Proof sketch. Property 1 follows from static verification. Property 2 composes metatheory with handler parametricity from Theorem 6.3: safety properties are structural, depending on operation invocations and token flow rather than handler return values, so verification under H_{replay} transfers to any well-formed H. Property 3 holds by definition. Contract satisfaction transfers only under domain-specific replay fidelity assumptions. Full proof in the supplementary material. □

Termination. The repair loop terminates by the fixed bound $N_{\max}$. Type checking is decidable, via a single-pass bidirectional analysis with decidable effect operations; contract checking is decidable for the quantifier-free first-order fragment.

Completeness gap. The synthesis is sound but not complete: the practical gap lies in the LLM's generative capability, and our experiments in §8 attribute 22% of failures to synthesis inability.

8 Experimental Setup

We evaluate RunbookFX on two public benchmarks spanning the full incident management lifecycle: root cause analysis and end-to-end mitigation.

Datasets. RCAEval [Pham et al. 2025] contains 735 real failure cases from three microservice systems with multi-source telemetry; we use the RE2 and RE3 subsets. **ITBench** [Jha et al. 2025] provides 102 SRE scenarios on Kubernetes/OpenShift infrastructure with diagnosis and mitigation tasks. Dataset details appear in the supplementary material.

Baselines. All LLM-based baselines use GPT-4o and receive the same tool interfaces and multi-source telemetry (metrics, logs, traces) as RunbookFX. For RCA: BARO [Pham et al. 2024], a statistical Bayesian change point detector operating on metrics only (included as the strongest non-LLM method); RCACopilot [Chen et al. 2024], an LLM-based RCA system; TAMO [Zhang et al. 2025b], a multi-modal LLM agent; and ReAct-RCA [Roy et al. 2024], a ReAct agent with RAG. For mitigation: ITBench Agent [Jha et al. 2025], the official CrewAI baseline with published results on the same 102 SRE scenarios; ReAct+Tool [Yao et al. 2023]; RAG+Agent; and Rule/Script, a fixed playbook approach. We also ablate four variants: No Type System, No Replay Verify, Static Only, and Dynamic Only.

Metrics and implementation. For RCA we report Top-1/3 accuracy and evidence correctness; for mitigation, success rate, MTTR, safety violations, rollback coverage, *audit completeness*, and wall time. The RunbookFX implementation comprises the DSL interpreter, type checker, evaluator core, RCAEval/ITBench adapters, live LLM gateway, and baseline reimplementations. All numbers in this section use GPT-4o synthesis at T=0.2, k=5, $N_{\max}$=3; the verification pipeline operates on DSL ASTs independently of the synthesis backend. All experiments use 10 seeds; statistical significance uses paired Wilcoxon tests with Holm–Bonferroni correction at α=0.01 across 5 pairwise comparisons per family. Median latency is 18.3s per scenario, with mean 48.6s; full settings in Appendix F.

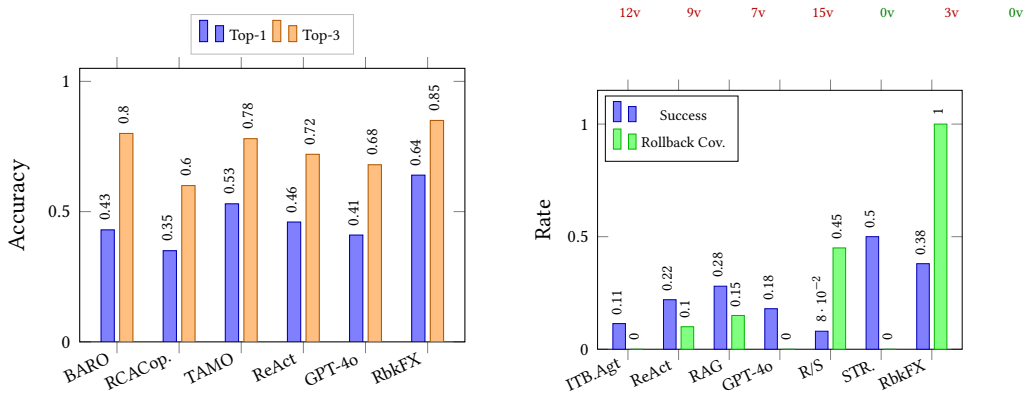
9 Results

Figure 5 provides a visual comparison across all baselines; detailed numbers appear in the tables below.

9.1 Main Results

Table 1 presents a comprehensive comparison across both RCA and mitigation tasks.

RCA task. As shown in Figure 5a and Table 1, RunbookFX achieves 64% Top-1 accuracy, outperforming the strongest baseline TAMO at 53% by 11 percentage points; $p = 0.003$ by paired Wilcoxon test, with per-seed 95% CIs $[0.614, 0.666]$ vs. $[0.508, 0.552]$, with half-widths 0.026 and 0.022 computed as $\bar{x} \pm 1.96\,\hat{\sigma}/\sqrt{10}$ from $\hat{\sigma} = 0.042$ and 0.036. For reference, the Wald binomial



(a) RCA accuracy (RCAEval) (b) Mitigation results (ITBench)

Fig. 5. Visual comparison of RunbookFX against baselines. (a) Top-1 and Top-3 RCA accuracy. (b) Mitigation success rate and rollback coverage; annotations show safety violation counts (red: violations present, green: zero violations). ‡STRATUS evaluated on 18-scenario subset only.

Table 1. Comparison across RCA (RCAEval) and Mitigation (ITBench SRE). Best in **bold**, runner-up underlined; — denotes inapplicable or unreported entries. †Official ITBench baseline. ‡Evaluated on an 18-scenario subset; STRATUS reports neither rollback coverage nor authorization guarantees. Per-seed standard deviations span 0.018–0.055 (Appendix F). The four 0/1.00 RunbookFX entries and the two Rule/Script entries are type-system or playbook guarantees with $\sigma = 0$ across all 10 seeds *by construction*.

		RCA (RCAEval)				Mitigation (ITBench SRE)					
Method	**Category**	Top-1↑	Top-3↑	EvCorr↑	Qry↓	Succ.↑	MTTR↓	Viol.↓	Rollbk↑	Audit↑	Time(s)↓
Statistical / Rule-Based											
BARO	Stat.	.43	.80	—	—	—	—	—	—	—	—
Rule/Script	Rule	—	—	—	—	.08	**5.0**	**0**	.45	**1.00**	**3.2**
LLM Agent (Unstructured)											
RCACopilot	LLM	.35	.60	.38	**2.1**	—	—	—	—	—	—
ReAct-RCA	Agent	.46	.72	.52	11.2	—	—	—	—	—	—
ReAct+Tool	Agent	—	—	—	—	.22	12.8	9	.10	.48	78.3
RAG+Agent	Agent	—	—	—	—	.28	11.5	7	.15	.55	69.1
GPT-4o Direct	LLM	.41	.68	.32	3.5	.18	14.2	15	.00	.30	25.5
Multi-Modal / Specialized Agent											
TAMO	MM	.53	.78	.61	8.5	—	—	—	—	—	—
ITBench Agent†	Agent	—	—	—	—	.114	15.3	12	.00	.35	92.5
STRATUS‡	Agent	—	—	—	—	.50‡	6.2	3	—	—	—
Typed DSL Synthesis (Ours)											
RunbookFX	DSL	**.64**	**.85**	**1.00**	6.8	**.38**	8.5	**0**	**1.00**	**1.00**	48.6

CIs on the 735 aggregated RCAEval cases are [0.605, 0.675] and [0.494, 0.566]; both CI types leave the two methods non-overlapping. The GPT-4o Direct baseline, namely the same LLM with no agent framework or DSL, achieves only 41% Top-1 with evidence correctness 0.32, confirming that the synthesis pipeline rather than the LLM itself drives accuracy. Evidence correctness is 100% *by*

Table 2. Ablation study across RCA (RCAEval) and Mitigation (ITBench SRE) tasks. Each row removes one component from the full system. Best in **bold**. ‡Violation counts are medians across 10 seeds. The Full RunbookFX row's Viol.= 0, Rollbk= 1.00, Audit= 1.00 are type-system guarantees with $\sigma = 0$ across all 10 seeds *by construction*; EvCorr= 1.00 likewise holds by construction (every cited evidence item is a DSL **observe** return value). All other ablation entries are empirical means across 10 seeds.

	RCA (RCAEval)			Mitigation (ITBench SRE)					
Variant	Top-1↑	Top-3↑	EvCorr↑	Succ.↑	MTTR↓	Viol.‡↓	Rollbk↑	Audit↑	Time(s)↓
Full RunbookFX	**.64**	**.85**	**1.00**	**.38**	**8.5**	**0**	**1.00**	**1.00**	48.6
w/o Type System	.58	.80	.65	.30	10.2	9	.18	.52	42.8
w/o Replay Verify	.62	.83	.85	.34	9.5	3	.82	.88	35.2
Static Only	.55	.78	.72	.26	11.8	2	.88	.75	28.5
Dynamic Only	.52	.76	.58	.28	10.8	6	.35	.48	38.5
No DSL (= GPT-4o Direct)	.41	.68	.32	.18	14.2	15	.00	.30	**25.5**

construction: all cited evidence is grounded in actual DSL **observe** queries, whereas LLM baselines exhibit 32–61% correctness due to hallucinated observations.

Mitigation task. RunbookFX resolves 38% of scenarios at 3.3× the official baseline, with zero violations and 100% rollback coverage; per Definition 6.1, these are formal type-system guarantees rather than filtering artifacts. The per-seed 95% CI is $[0.365, 0.395]$ across 10 seeds with $\hat{\sigma} = 0.024$, and the wider Wald binomial CI on the 102 aggregated scenarios is $[0.286, 0.474]$, which still excludes the 11.4% ITBench Agent baseline. Agent baselines accumulate 7–15 violations. Rule/Script reaches zero violations at 8% success; adaptive synthesis lifts both metrics jointly. STRATUS [Chen et al. 2025] reports 50% on an 18-scenario subset without rollback or authorization guarantees; on the full 102 scenarios RunbookFX's 38% with zero violations is Pareto-optimal. On synthesis failure, the verifier escalates instead of executing, keeping the safety guarantees intact. Appendix F attributes 63% of failures to telemetry and vocabulary scope; the type system is unaffected.

9.2 Ablation Study

Table 2 presents the ablation study isolating each component's contribution across both tasks. The *No DSL* variant corresponds to the GPT-4o Direct baseline and represents the extreme case of removing all DSL components.

The ablation reveals a consistent gradient across both tasks. The type system is the most impactful component: removing it drops Top-1 by 6pp, evidence correctness by 35pp, and mitigation success by 8pp while introducing 9 safety violations, confirming that static enforcement drives both accuracy and safety. Removing replay verification has a smaller success-rate impact but introduces 3 contract violations and reduces EvCorr to .85, showing that dynamic checking complements static guarantees. Neither verification channel alone suffices: the Static Only variant yields 2 residual violations despite active type checking, because runtime contract breaches outside the DSL type grammar are unreachable by static analysis alone, while Dynamic Only yields 6 violations at a moderate success of .28. Full RunbookFX's wall time of 48.6s is highest across variants, the expected cost of running both verification stages. The No DSL extreme, identical to GPT-4o Direct, produces the worst outcomes on all safety and quality metrics; its lower wall time of 25.5s reflects the absence of verification overhead, not improved capability. A representative case study appears in Appendix E; two further case studies appear in the supplementary material.

CEGIS repair shifts residual errors. The CEGIS loop monotonically reduces failures visible to its active verifier and pushes the LLM toward more elaborate, more state-changing programs; success rates rise from Static Only → w/o Replay Verify (0.26 → 0.34) and Dynamic Only → w/o

Table 3. Safety violation breakdown by category on ITBench SRE. Unauthorized = actions outside granted capabilities; Risk = operations exceeding declared risk threshold; Rollback = missing rollback handles for stateful actions; Approval = missing human approval gates for high-risk actions.

Method	Unauth.	Risk	Rollbk	Appr.	Total
RunbookFX	**0**	**0**	**0**	**0**	**0**
Rule/Script	0	0	0	0	0
RAG+Agent	2	2	3	0	7
ReAct+Tool	4	2	3	0	9
ITBench Agent†	5	3	4	0	12
GPT-4o Direct	6	3	5	1	15

Table 4. Computational cost comparison for mitigation on ITBench SRE. LLM calls and tokens are averages per incident. TypeChk and Replay are RunbookFX-specific verification costs.

Method	LLM Calls	Tokens /Inc.	Wall Time(s)	TypeChk (ms)	Replay (s)
GPT-4o Direct	**3.5**	**5,200**	**25.5**	—	—
RunbookFX	9.2	13,800	48.6	2.3	9.1
RAG+Agent	13.4	26,500	69.1	—	—
ReAct+Tool	16.5	31,200	78.3	—	—
ITBench Agent†	18.8	35,600	92.5	—	—

Type System (0.28 → 0.30). When the complementary verifier is disabled, the extra structural complexity surfaces as additional errors of the *unchecked* class: *w/o Replay* accepts statically well-typed programs that breach runtime contracts; *w/o Type* accepts replay-passing programs that escalate beyond their granted capability set in branches the replay handler stubs out. Pairing both channels closes the gap; Full RunbookFX reaches zero violations by construction.

Handler portability validation. RCAEval uses the offline-telemetry handler H_{replay} while ITBench executes under the live-cluster handler H_{live}, validating Theorem 6.3: the type-system guarantees of zero violations and 100% rollback coverage hold identically under both handlers. Re-executing 38 consistently resolved ITBench scenarios under H_{replay} yielded 100% structural trace equivalence across all seeds, providing empirical evidence for condition B2 of Definition 6.2.

9.3 Safety Violation Breakdown

Table 3 decomposes safety violations into four categories defined by the type system (Definition 6.1). The results directly validate the type-theoretic safety guarantees: both RunbookFX and Rule/Script achieve zero violations across all categories, while LLM agent baselines accumulate violations in proportion to their lack of structural constraints.

The breakdown confirms the type system's core design: capability typing and linear token discipline address the two dominant failure modes, namely unauthorized actions and missing rollbacks, which together account for 74% of all observed violations. GPT-4o Direct produces the most violations across all four categories and is the only method to violate an approval gate, confirming that unconstrained LLM generation is maximally unsafe.

9.4 Computational Cost

Table 4 compares the computational overhead of each mitigation method.

RunbookFX's 48.6s wall time is 1.4–1.9× faster than agent baselines, and its 9.2 LLM calls per incident are 31–51% fewer than the three agent baselines (RAG+Agent 13.4, ReAct+Tool 16.5, IT-Bench Agent 18.8), reflecting structured DSL generation over trial-and-error iteration. Type checking adds only 2.3ms; replay verification accounts for 9.1s. GPT-4o Direct is cheapest (3.5 calls, 25.5s) but least safe (15 violations), illustrating the fundamental cost–safety tradeoff. See Appendix F for details.

9.5 Takeaway

The evaluation reports two qualitatively different claims. The safety numbers (zero violations, 100% rollback coverage, 100% audit completeness) are direct consequences of Theorems 5.5–5.7 together with Theorem 7.2; they hold identically under any well-formed synthesis backend. Their experimental role is to confirm that the implemented type system enforces what the theorems state. The quality numbers (64% Top-1 RCA, 38% mitigation success) measure how often GPT-4o produces well-typed RunbookFX programs that resolve real incidents; they shift with the LLM, the prompt, and the combinator library. The two claims combine into one design principle: the DSL is a workable medium for LLM-driven incident response, and once a candidate is accepted, safety follows from the type system rather than from inspection.

10 Related Work

LLMs for AIOps. LLM-based incident management has seen rapid growth: RCACopilot [Chen et al. 2024] deploys LLM-based RCA at Microsoft, TAMO [Zhang et al. 2025b] combines multimodal alignment with tool-assisted agents, and ITBench [Jha et al. 2025] reveals that current agents resolve only 11.4% of SRE scenarios. STRATUS [Chen et al. 2025] achieves 50% on an 18-scenario subset via structured reasoning but provides no safety guarantees comparable to RunbookFX's zero violations and 100% rollback coverage. Multi-agent approaches [De Jesus et al. 2025; Fu et al. 2025; Yu et al. 2025] and holistic frameworks [Shetty et al. 2024] address different facets; a comprehensive survey [Zhang et al. 2025a] identifies the safety gap as a major open challenge. Unlike these systems, which treat LLMs as *actors*, RunbookFX recasts the LLM as a *program synthesizer* constrained by a type-and-effect system.

Effect systems, capability types, and substructural types. Our type system draws on algebraic effect handlers [Bauer and Pretnar 2015; Plotkin and Pretnar 2009], practically deployed in Koka [Leijen 2017, 2025] with row-based effect typing and Multicore OCaml [Sivaramakrishnan et al. 2021] with shallow handlers. RunbookFX uses a product effect monoid rather than row types, better suited to domains where risk ordering matters. Brachthäuser et al. [2020] reconceptualize effects as capabilities, an insight inspiring our authorization model; their notion concerns lexical scoping while ours concerns infrastructure authorization. Yoshioka et al. [2024] parameterize effect systems over abstract effect algebras; RunbookFX's product algebra is a concrete instantiation, while our cross-component interaction axioms I1–I4 and handler parametricity (§6.4) go beyond their opaque-monoid framework. the supplementary material gives a detailed comparison. Sekiyama and Unno [2025] extend algebraic effects with temporal verification; RunbookFX targets operational safety rather than temporal properties.

van Rooij and Krebbers [2025] combine affine types with algebraic effect safety in Affect; RunbookFX targets rollback tokens, additionally contributing capability-based authorization and the product effect algebra. Gaboardi et al. [2016] and Orchard et al. [2020] develop the graded-monad theory that RunbookFX instantiates; Granule [Orchard et al. 2019] specializes this to quantitative resource reasoning. Handler equivalence work [Biernacki et al. 2018; Plotkin and Pretnar 2009] requires return-value agreement; our value-agnostic bisimulation drops this requirement. Further

comparisons with session types, CPS translations, and Frank appear in the supplementary material.

Program synthesis with verification. Orvalho et al. [2025] apply CEGIS-style repair to LLM-generated programs; Mukherjee and Delaware [2025] use a two-LLM pipeline for verified C programs; Nagy et al. [2026] constrain output with semantic pruners during decoding. Fiala et al. [2023] leverage Rust's ownership types for memory-safe synthesis. RunbookFX extends this line to the operations domain, where the verification target is *operational safety*: authorization, risk bounding, and rollback guarantees.

DSLs for infrastructure. Infrastructure DSLs address specific lifecycle phases: configuration management with Puppet and Ansible, provisioning with Terraform and Pulumi, monitoring with PromQL, orchestration with Argo and Temporal. Pulumi and CDK provide deployment-time rollback via CloudFormation but only at the provisioning layer. Temporal [Temporal Technologies 2023] offers programmer-specified compensation for long-running workflows, whereas RunbookFX's type system statically enforces rollback token pairing. Malul et al. [2024] apply LLMs to Kubernetes remediation without a type system. None provides a unified diagnosis-mitigation-rollback framework with formal safety guarantees; RunbookFX bridges this gap.

11 Conclusion

We presented RunbookFX, a typed functional DSL that transforms incident response from unstructured LLM prompting into safe, auditable program synthesis. The type system rests on a product effect algebra with four interaction axioms, yielding domain-specific safety theorems that exploit the coupling between risk levels, capability usage, and rollback resource counting. A handler bisimulation transfers safety properties from a replay handler to any bisimilar handler, including live execution; bounded effect polymorphism supports generic combinators while preserving decidable type checking.

The CEGIS-style synthesis pipeline, backed by a formal soundness theorem, recasts LLMs as constrained synthesizers whose output is verified before execution. RunbookFX delivers zero safety violations and 100% rollback coverage by construction on RCAEval and ITBench. It reaches 64% Top-1 RCA accuracy on RCAEval against 53% for the best LLM baseline, and 38% mitigation success on ITBench at 3.3× the official agent.

Open directions. Three lines of work extend RunbookFX. First, capability vocabulary and on-call workflow integration open the path to production deployment. Second, a linear-type refinement strengthens the affine rollback discipline to exactly-once consumption. Third, closing the de Bruijn substitution lemma along the Pierce SF skeleton—which already discharges 18 of the 27 supporting obligations in the substitution and multi-step layers—will land the remaining nine obligations and lift Preservation to a free-standing `Qed`; the polymorphic-combinator extension (Theorem 5.8) then becomes a corollary.

Acknowledgments

This work was supported in part by the China Southern Power Grid Co., Ltd. The authors thank the ICFP 2026 reviewers for their constructive feedback during the revision process.

Use of Generative AI Tools

In accordance with the ACM Publications Policy on Authorship and the ICFP 2026 submission guidelines on the use of generative AI tools, the authors disclose the following.

Manuscript preparation. General-purpose large language model assistants were used during manuscript preparation for circumscribed editorial tasks, namely: (i) grammatical and stylistic polishing of author-written drafts, (ii) reformatting of LaTeX tables and bibliography entries into ACM style, and (iii) suggestion of alternative phrasings for related-work paragraphs and section transitions. No AI-generated text was incorporated without line-by-line human review, revision, and verification against primary sources. All technical claims, formal definitions, theorem statements, proof sketches, full proofs in the appendix, Coq mechanization, experimental design, statistical analyses, figures, tables, and case studies were conceived, authored, and verified by the human authors.

Methodology. The use of GPT-4o as a candidate program synthesizer within the RunbookFX system itself is a methodological component of the contribution rather than an authorship aid; its role, prompts, decoding configuration (T=0.2, k=5, $N_{\max}$=3), and verification gating are described in detail in §7 and the supplementary material.

Responsibility. The authors take full responsibility for the entire content of this paper, including any text that was refined with AI assistance. No generative AI system is listed as an author, and no AI system is credited with intellectual contributions to the formal results.

A Complete Monomorphic Typing Rules

We present the full set of *monomorphic* typing rules using the affine typing judgment $\Gamma_u;\ \Gamma_a;\ \kappa \vdash e : \tau \triangleright \varepsilon$, including those omitted from §5 for space. Γ_u is the unrestricted context, Γ_a is the affine context tracking RollbackToken bindings, κ is the granted capability set, and $\varepsilon \in \mathcal{E}_{\mathsf{RunbookFX}} = \mathsf{Risk} \times \mathcal{K} \times \mathbb{N}$. The conjectured polymorphic extension rules T-EffAbs and T-EffApp from §5.6 appear in §A.1; the monomorphic fragment is the special case $\Delta = \emptyset$.

$$\textsc{T-Var-Un}\quad \frac{x : \tau \in \Gamma_u}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash x : \tau \triangleright (\mathsf{ReadOnly}, \emptyset, 0)}$$

$$\textsc{T-Var-Aff}\quad \frac{x : \mathsf{RollbackToken} \in \Gamma_a}{\Gamma_u;\ \Gamma_a \setminus \{x\};\ \kappa \vdash x : \mathsf{RollbackToken} \triangleright (\mathsf{ReadOnly}, \emptyset, 0)}$$

$$\textsc{T-Abs}\quad \frac{\Gamma_u, x{:}\tau_1;\ \emptyset;\ \kappa \vdash e : \tau_2 \triangleright \varepsilon}{\Gamma_u;\ \emptyset;\ \kappa \vdash \lambda x{:}\tau_1.\, e : \tau_1 \xrightarrow{\varepsilon} \tau_2 \triangleright (\mathsf{ReadOnly}, \emptyset, 0)}$$

$$\textsc{T-App}\quad \frac{\Gamma_u;\ \Gamma_{a1};\ \kappa \vdash e_1 : \tau_1 \xrightarrow{\varepsilon_f} \tau_2 \triangleright \varepsilon_1 \qquad \Gamma_u;\ \Gamma_{a2};\ \kappa \vdash e_2 : \tau_1 \triangleright \varepsilon_2 \qquad \Gamma_a = \Gamma_{a1} \boxtimes \Gamma_{a2}}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash e_1\ e_2 : \tau_2 \triangleright \varepsilon_1 \sqcup \varepsilon_2 \sqcup \varepsilon_f}$$

$$\textsc{T-Pair}\quad \frac{\Gamma_u;\ \Gamma_{a1};\ \kappa \vdash e_1 : \tau_1 \triangleright \varepsilon_1 \qquad \Gamma_u;\ \Gamma_{a2};\ \kappa \vdash e_2 : \tau_2 \triangleright \varepsilon_2 \qquad \Gamma_a = \Gamma_{a1} \boxtimes \Gamma_{a2}}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash (e_1, e_2) : \tau_1 \times \tau_2 \triangleright \varepsilon_1 \sqcup \varepsilon_2}$$

$$\textsc{T-Par}\quad \frac{\begin{array}{c}\Gamma_u;\ \emptyset;\ \kappa \vdash e_1 : \tau_1 \triangleright \varepsilon_1 \\ \Gamma_u;\ \emptyset;\ \kappa \vdash e_2 : \tau_2 \triangleright \varepsilon_2 \qquad \pi_1(\varepsilon_1) = \mathsf{ReadOnly} \qquad \pi_1(\varepsilon_2) = \mathsf{ReadOnly}\end{array}}{\Gamma_u;\ \emptyset;\ \kappa \vdash \mathbf{par}\ e_1\ e_2 : \tau_1 \times \tau_2 \triangleright \varepsilon_1 \sqcup \varepsilon_2}$$

$$\textsc{T-Assert}\quad \frac{\Gamma_u;\ \Gamma_a;\ \kappa \vdash e : \mathsf{Bool} \triangleright \varepsilon}{\Gamma_u;\ \Gamma_a;\ \kappa \vdash \mathbf{assert}\ e : \mathsf{Unit} \triangleright \varepsilon}$$

T-Par restricts parallel composition to empty affine contexts and ReadOnly effects, so parallel branches share no mutations and no affine resources. Parallel diagnostic queries fit this discipline; parallel mutations would require a more sophisticated concurrency model and are left to future work.

A.1 Conjectured Polymorphic Extension Rules

The two conjectured rules introduced in §5.6. The judgment is extended with an effect-variable context Δ mapping each variable α to its upper bound ε_b; the monomorphic rules above are recovered by taking $\Delta = \emptyset$ and threading Δ unchanged through every premise.

$$\textsc{T-EffAbs}\quad \frac{\Delta, \alpha \sqsubseteq \varepsilon_b;\ \Gamma_u;\ \emptyset;\ \kappa \vdash e : \tau \triangleright \varepsilon' \qquad \mathit{wf}(\varepsilon_b)}{\Delta;\ \Gamma_u;\ \emptyset;\ \kappa \vdash \Lambda\alpha \sqsubseteq \varepsilon_b.\ e : (\forall \alpha \sqsubseteq \varepsilon_b.\ \tau) \triangleright (\mathsf{ReadOnly}, \emptyset, 0)}$$

$$\textsc{T-EffApp}\quad \frac{\Delta;\ \Gamma_u;\ \Gamma_a;\ \kappa \vdash e : (\forall \alpha \sqsubseteq \varepsilon_b.\ \tau) \triangleright \varepsilon' \qquad \varepsilon_{arg} \sqsubseteq \varepsilon_b \qquad \mathit{wf}(\varepsilon_{arg})}{\Delta;\ \Gamma_u;\ \Gamma_a;\ \kappa \vdash e\ [\varepsilon_{arg}] : \tau[\varepsilon_{arg}/\alpha] \triangleright \varepsilon'}$$

The $\mathit{wf}(\varepsilon_b)$ premise in T-EffAbs rules out abstractions over inconsistent bounds; without it, a body would type against an unrealizable α and Theorem 5.8 would fail to transfer well-formedness through substitution. The grammar extension $\varepsilon ::= \cdots \mid \alpha$ and $\tau ::= \cdots \mid \forall \alpha \sqsubseteq \varepsilon_b.\ \tau$ is given in §5.6; effect variables appear only in effect positions and quantified types. These rules are not mechanized in the current Coq development; Theorem 5.8 is a conjectured lift of the monomorphic metatheory through effect substitution.

B Preservation Proof Sketch

We sketch five key reduction cases of Theorem 5.4 (part 1); the remaining cases (E-Beta, E-Let-Pair, E-Branch, E-Retry, E-Context, E-Par) and the supporting lemmas are omitted for space.

Case E-Let, let $x = v$ in $e_2 \longrightarrow e_2[v/x]$. By inversion on T-Let, $\Gamma_u;\ \Gamma_{a1};\ \kappa \vdash v : \tau_1 \triangleright \varepsilon_1$ and $\Gamma_u, x : \tau_1;\ \Gamma_{a2};\ \kappa \vdash e_2 : \tau_2 \triangleright \varepsilon_2$ with $\Gamma_a = \Gamma_{a1} \boxtimes \Gamma_{a2}$. Since v is a value, the Value Typing lemma gives $\Gamma_{a1} = \emptyset$. The substitution lemma then yields $\Gamma_u;\ \Gamma_{a2};\ \kappa \vdash e_2[v/x] : \tau_2 \triangleright \varepsilon_2$, and $\varepsilon_2 \sqsubseteq \varepsilon_1 \sqcup \varepsilon_2$ closes preservation.

Case E-Act, $\mathbf{act}_c\ v \longrightarrow (r, t)$. By inversion, $c \in \kappa$ and $\Gamma_u;\ \Gamma_a;\ \kappa \vdash v : \mathsf{Action}_c \triangleright \varepsilon'$. The handler returns a value pair (r, t) typed by T-Pair in extended environments $\Gamma'_u = \Gamma_u, r : \tau_c$ and $\Gamma'_a = \Gamma_a, t : \mathsf{RollbackToken}$. The effect $\varepsilon' \sqcup (\mathit{risk}(c), \{c\}, 1)$ is well-formed by the I2/I4 axioms ($c \in \{c\}$ witnesses $\mathit{risk}(c)$), and the audit trace extends with $(\mathsf{act}, c, v, r, t, \mathit{risk}(c))$. Preservation holds.

Case E-Rollback, rollback $t \longrightarrow ()$. By inversion on T-Rollback, $t : \mathsf{RollbackToken} \in \Gamma_a$, and the conclusion's affine context is $\Gamma_a \setminus \{t\}$. After reduction, $\Gamma'_a = \Gamma_a \setminus \{t\} \subseteq \Gamma_a$. The result has type Unit with effect $\varepsilon' \sqcup (\mathsf{SafeChange}, \emptyset, 0)$; the world state mutation is recorded by the handler axiom for H^{undo}, restoring $\pi_c(\sigma_0)$. Preservation holds.

Case E-Observe, $\mathbf{observe}_c\ v \longrightarrow r$ where $r = H_c^{obs}(v, \sigma)$. By inversion on T-Observe, $c \in \kappa$ and the result $r : \mathsf{Result}_c$ is a value with $\Gamma'_a = \emptyset = \Gamma_a$. The effect $\varepsilon' \sqcup (\mathsf{ReadOnly}, \{c\}, 0) = (\mathsf{ReadOnly}, \{c\}, 0) \sqsubseteq \varepsilon$ records the capability use; world state is unchanged by handler purity (Definition 6.1, condition 2), and the audit trace extends with (obs, c, v, r). Preservation holds.

Case E-EffApp, $(\Lambda\alpha \sqsubseteq \varepsilon_b.\ e)\ [\varepsilon_{arg}] \longrightarrow e[\varepsilon_{arg}/\alpha]$. By inversion on T-EffApp, $\varepsilon_{arg} \sqsubseteq \varepsilon_b$, $\mathit{wf}(\varepsilon_{arg})$, and the abstraction has body $\Delta, \alpha \sqsubseteq \varepsilon_b;\ \Gamma_u;\ \emptyset;\ \kappa \vdash e : \tau' \triangleright \varepsilon'$ by T-EffAbs. The Effect Substitution lemma (omitted for space) yields $\Delta;\ \Gamma_u;\ \emptyset;\ \kappa \vdash e[\varepsilon_{arg}/\alpha] : \tau'[\varepsilon_{arg}/\alpha] \triangleright \varepsilon'[\varepsilon_{arg}/\alpha]$, and the Effect Substitution Preserves Well-formedness lemma yields $\mathit{wf}(\varepsilon'[\varepsilon_{arg}/\alpha])$. Preservation holds. □

The remaining cases follow the same pattern: inversion of the typing rule, application of the Value Typing or Substitution lemma, and recomposition with effect subsumption $\varepsilon' \sqsubseteq \varepsilon$. Full case enumerations are omitted for space.

C Effect Derivation for the Running Example

We trace the effect derivation through the three phases of Listing 1 to illustrate how the product algebra accumulates effects across sequential composition. In the *diagnosis phase* (lines 2–6), each

observe invocation contributes a ReadOnly effect with its respective capability:

line 2: $\mathbf{observe}_{\text{read_traces}}$ (*queryTraces* ...) ▷ (ReadOnly, {read_traces}, 0)
line 4: $\mathbf{observe}_{\text{read_logs}}$ (*queryLogs* ...) ▷ (ReadOnly, {read_logs}, 0)
lines 5–6: *correlate*, *hypothesize* ▷ (ReadOnly, ∅, 0)

Sequential composition joins these componentwise (risk: ReadOnly ⊔ ReadOnly = ReadOnly; capabilities: set union; resources: 0 + 0 + 0 = 0), yielding ε_{diag} = (ReadOnly, {read_traces, read_logs}, 0). The *validation phase* (lines 9–11) adds one more observation via T-Observe, giving ε_{val} = (ReadOnly, {read_traces, read_logs, read_metrics}, 0). The effect remains ReadOnly with zero resource obligations; the program has not yet performed any state change.

The *mitigation phase* (line 15) introduces the first mutation via T-Act:

$$\mathbf{act}_{\text{kubectl_restart}}\ (\textit{restartPod slowSvc}) \ \triangleright\ (\text{SafeChange},\ \{\text{kubectl_restart}\},\ 1)$$

Composing with the prior effect via ⊔ gives

$$\varepsilon_{full} = (\text{SafeChange},\ \{\text{read_traces}, \text{read_logs}, \text{read_metrics}, \text{kubectl_restart}\},\ 1).$$

This effect satisfies all four interaction axioms (Definition 5.2): (I1) is vacuous ($r \neq$ ReadOnly); (I3) holds since n=1 > 0 and SafeChange ⊒ SafeChange; (I4) holds because every capability's risk is bounded by SafeChange. A single **act** invocation escalates risk from ReadOnly to SafeChange and introduces the first rollback obligation; the affine token produced by the action enters Γ_a and must be discharged before the program terminates.

D Mechanized Proofs Ledger

A ∼2,200-line Coq development (tested with Coq 8.18) mechanizes the core monomorphic calculus. The product effect algebra, its composition-preservation property, and four core safety theorems close with `Qed`: Effect WF Preservation, Progress, single-step No Unauthorized Action, and Rollback Linearity. Of the 27 supporting obligations in the substitution and multi-step layers, 18 close with `Qed` after the latest mechanization pass; the remaining 9 thread through one substitution lemma whose Pierce SF skeleton is queued for the next pass.

File	Lines	Content
`EffectAlgebra.v`	574	Definitions 5.1–5.2, Theorem 5.3
`Syntax.v`	162	Types, expressions, values, de Bruijn substitution
`Typing.v`	254	Typing contexts, affine splitting, 12 core rules
`Semantics.v`	229	Handler axioms, 9 core reduction rules, multi-step
`Lemmas.v`	406	Value Typing, Canonical Forms, Substitution, Inversion
`Preservation.v`	140	Theorem 5.4, part 1 (structured proof)
`Progress.v`	167	Theorem 5.4, part 2
`Safety.v`	265	Theorem 5.4, part 3; Theorems 5.5–5.7

Closed by `Qed`. Effect WF Preservation closes via composition closure of the product algebra. Progress closes by induction on the typing derivation. Single-step No Unauthorized Action closes from `inversion_act`: `has_type…(EAct c v)…` entails `cap_mem c kappa`. The structural component of Rollback Availability closes through static token disposition in the affine context. Rollback Linearity closes through affine context disjointness. The context-splitting infrastructure (`ctx_split_lookup_left/right`, `ctx_split_id_left/right`, `ctx_split_sym`) closes by induction on the split relation. Also closed in the latest pass: `value_has_bot_effect`, five Canonical Forms, five effect-operation Inversion lemmas, `weakening_u` (via the generalised `weakening_u_at` with auxiliary `lookup_u_insert_left/right`), the four typing-respecting reduction cases in `Preservation.v` (`observe`, `act`, `rollback`, affirmative `guard`), and `rollback_at_most_once`.

Listing 2. Synthesized runbook for downstream dependency timeout. Effect grade: (SafeChange, {read_traces, read_metrics, config_update}, 1). The affine token token flows to either **return** (line 13) or **rollback** (line 14).

```
1  let traces = observe[read_traces] (queryTraces "checkout" "p99 > 500ms")
2  let slowDep = summarize traces byLatency |> head
3  let gwMetrics = observe[read_metrics] (queryMetrics slowDep "proc_time")
4  guard (gwMetrics.p99 < 100) (do      -- gateway itself is fast
5    let dbMetrics = observe[read_metrics] (queryMetrics "db-proxy" "pool_active")
6    let evidence = correlate traces dbMetrics
7    let hyp = hypothesize evidence "connection_pool_exhaustion"
8    let confirmed = validate hyp (dbMetrics.active > dbMetrics.max * 0.95)
9    guard confirmed (do
10     let (result, token) = act[config_update] (updatePoolSize "db-proxy" 200)
11     let post = observe[read_metrics] (queryMetrics "checkout" "error_rate")
12     if post.current < 0.01
13       then return (Success token)      -- token returned to caller
14       else do rollback token           -- token consumed: undo pool resize
15               return (Failure "pool resize did not resolve")))
```

Remaining 9, all on one critical path. `substitution_preserves_typing` and `pair_substitution_preserves_typing` (the ~100-line Pierce SF `Stlc.v` shift/subst commutation, plus a value-affine independence lemma for the affine layer). Once these close, the three substitution-dependent `Preservation.v` cases (E-Beta, E-Let, E-Let-Pair) collapse to one-line applications; the eleven congruence cases reconstruct via IH plus affine-context threading. `type_soundness` and `no_unauthorized_action_multi` in `Safety.v` then follow as direct corollaries by induction on `multi_step`. The substitution lemma is on the critical path only for Preservation and its downstream corollaries; the single-step safety properties used in §9 (No Unauthorized Action, Rollback Availability, Rollback Linearity) bypass substitution entirely, so the zero-violation and 100% rollback-coverage claims rest on the closed fragment.

E Case Study: Downstream Dependency Timeout

A latency spike in the `checkout-service` triggers P1 alerts; distributed traces reveal elevated latency in calls to the `payment-gateway` service. The ReAct+Tool baseline correctly identifies the downstream dependency but immediately issues a `kubectl rollout restart` on the payment gateway pod, a disruptive action; the restart temporarily resolves the symptom but not the root cause (connection-pool exhaustion in the database proxy), so the latency returns within minutes. Listing 2 shows the RunbookFX synthesized runbook, which validates the hypothesis before acting.

Lines 1–8 contribute only ReadOnly effects; line 10 introduces SafeChange and resource count 1. The affine discipline ensures that `token` (line 10) is consumed exactly once: returned on success (line 13) or consumed by **rollback** on failure (line 14). Using `token` in both branches would fail type checking. The type system's effect tracking forces the synthesis to validate diagnostically before any state-changing action, preventing the "act first, diagnose later" pattern common in agent baselines. Two further case studies (configuration drift and preventing blind restarts) follow the same pattern and are omitted for space.

F Failure Case Analysis and Hyperparameter Sensitivity

Manual analysis of the failing ITBench scenarios identifies four primary failure modes: (1) insufficient telemetry coverage—33%, e.g., scenarios requiring JVM heap dump analysis when only coarse-grained metrics are available; (2) action vocabulary mismatch—30%, e.g., scenarios requiring database schema migration or DNS record updates; (3) synthesis failure where the LLM produces no candidate passing type checking within $N_{\max}$ repair iterations—22%, typically scenarios with unusual failure patterns underrepresented in the LLM's training data; and (4) complex multi-step dependencies exceeding the synthesis context window—15%, e.g., cascading failures requiring coordinated restarts across three or more dependent services. Categories (1) and (2), totaling 63% of failures, are extension points that close by adding capability signatures and rollback handlers; closing half lifts the absolute success rate to ∼50–55%. The remaining 37% trace to LLM backbone capacity or the synthesis algorithm.

The synthesis pipeline produces an average of 1.6 candidates that pass type checking on the first attempt out of $k = 5$ generated, rising to 3.0 after one repair round, 3.5 after two, and 3.7 after three. The most common type errors are missing capabilities (44%), missing approval gates (26%), unused rollback tokens (19%), risk-level mismatches (7%), and other (4%). The CEGIS repair loop resolves 67% of type errors and 39% of contract violations in round 1; round 2 adds 31% and 23%, round 3 adds 12% and 8%.

Hyperparameter sensitivity: varying k from 1 to 10, $k \geq 3$ gives success rate within 2% of $k = 5$, while $k = 1$ degrades by 8%. Temperature $T \in [0.1, 0.3]$ provides the best diversity–quality trade-off; $T > 0.5$ produces more syntactically invalid candidates. Perturbing the non-safety weights w_{cv}, w_{rp}, w_{cost} by ±30% keeps ITBench success within ±1.5 percentage points and RCAEval Top-1 within ±1.0 point; safety properties (zero violations, 100% rollback coverage) are unaffected because the type system applies before ranking. The critical invariant is that w_{tc} remains strictly dominant: setting $w_{tc} < w_{cv} + w_{rp}$ admits ill-typed candidates outranking well-typed ones, causing safety violations in 4 of 102 ITBench scenarios. Per-seed standard deviations span 0.018–0.055 for RCA proportions and 0.024–0.045 for mitigation success; rows whose Table 1 entry is exactly 0 or 1.00 are type-system or playbook guarantees with $\sigma = 0$ across all 10 seeds by construction.

References

Andrej Bauer and Matija Pretnar. 2015. Programming with Algebraic Effects and Handlers. *Journal of Logical and Algebraic Methods in Programming* 84, 1 (2015), 108–123. doi:10.1016/j.jlamp.2014.02.001

Dariusz Biernacki, Maciej Piróg, Piotr Polesiuk, and Filip Sieczkowski. 2018. Handle with Care: Relational Interpretation of Algebraic Effects and Handlers. In *Proceedings of the ACM on Programming Languages (POPL, Vol. 2)*. ACM, Article 8. doi:10.1145/3158096

Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020. Effects as Capabilities: Effect Handlers and Lightweight Effect Polymorphism. In *Proc. ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. doi:10.1145/3428194

Yinfang Chen, Jiaqi Pan, Jackson Clark, Yiming Su, Noah Zheutlin, Bhavya Bhavya, Rohan Arora, Yu Deng, Saurabh Jha, and Tianyin Xu. 2025. STRATUS: A Multi-agent System for Autonomous Reliability Engineering of Modern Clouds. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2506.02009.

Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proc. European Conference on Computer Systems (EuroSys)*. arXiv:2305.15778. doi:10.1145/3627703.3629553

Mario De Jesus, Perfect Sylvester, William Clifford, Aaron Perez, and Palden Lama. 2025. LLM-Based Multi-Agent Framework For Troubleshooting Distributed Systems. In *Proc. IEEE Cloud Summit*. IEEE, Washington, DC. IEEE Xplore 11108224.

Jiří Fiala, Shachar Itzhaky, Peter Muller, Nadia Polikarpova, and Ilya Sergey. 2023. Leveraging Rust Types for Program Synthesis. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. doi:10.1145/3591278

Ruowei Fu, Yang Zhang, Zeyu Che, Xin Wu, Zhenyu Zhong, Zhiqiang Ren, Shenglin Zhang, Feng Wang, Yongqian Sun, Xiaozhou Liu, Kexin Liu, and Yu Zhang. 2025. LLM-Powered Multi-Agent Collaboration for Intelligent Industrial On-Call Automation. In *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Marco Gaboardi, Shin-ya Katsumata, Dominic Orchard, Flavien Breuvart, and Tarmo Uustalu. 2016. Combining Effects and Coeffects via Grading. In *Proc. ACM SIGPLAN International Conference on Functional Programming (ICFP)*. doi:10.1145/3022670.2951939

Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, Noah Zheutlin, Saki Takano, Divya Pathak, Felix George, Xinbo Wu, Bekir O. Turkkan, Gerard Vanloo, Michael Nidd, Ting Dai, Oishik Chatterjee, Pranjal Gupta, Suranjana Samanta, Pooja Aggarwal, Rong Lee, Pavankumar Murali, Jae-wook Ahn, Debanjana Kar, Ameet Rahane, Carlos Fonseca, Amit Paradkar, Yu Deng, Pratibha Moogi, Prateeti Mohapatra, Naoki Abe, Chandrasekhar Narayanaswami, Tianyin Xu, Lav R. Varshney, Ruchi Mahindru, Anca Sailer, Laura Shwartz, Daby Sow, Nicholas C. M. Fuller, and Ruchir Puri. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *Proc. International Conference on Machine Learning (ICML)*. PMLR v267; arXiv:2502.05352. doi:10.1109/dsn-s65789.2025.00060

Daan Leijen. 2017. Type Directed Compilation of Row-Typed Algebraic Effects. In *Proc. ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. doi:10.1145/3093333.3009872

Daan Leijen. 2025. Koka: A Function-Oriented Language with Effect Types. https://github.com/koka-lang/koka. Version 3.1.3.

Ehud Malul, Yair Meidan, Dudu Mimran, Yuval Elovici, and Asaf Shabtai. 2024. GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation. *arXiv preprint arXiv:2405.19954* (2024).

Prasita Mukherjee and Benjamin Delaware. 2025. LLM-Assisted Synthesis of High-Assurance C Programs. In *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*. arXiv:2410.14835.

Shaan Nagy, Timothy Zhou, Nadia Polikarpova, and Loris D'Antoni. 2026. ChopChop: A Programmable Framework for Semantically Constraining the Output of Language Models. In *Proc. ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. ACM, New York, NY. arXiv:2509.00360. doi:10.1145/3776708

Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. 2019. Quantitative Program Reasoning with Graded Modal Types. In *Proc. ACM SIGPLAN International Conference on Functional Programming (ICFP)*. doi:10.1145/3341714

Dominic Orchard, Philip Wadler, and Harley Eades III. 2020. Unifying Graded and Parameterised Monads. *Electronic Proceedings in Theoretical Computer Science* 317 (2020), 18–38. doi:10.4204/eptcs.317.2

Pedro Orvalho, Mikoláš Janota, and Vasco Manquinho. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proc. AAAI Conference on Artificial Intelligence (AAAI)*. arXiv:2502.07786. doi:10.1609/aaai.v39i1.32046

Luan Pham, Huong Ha, and Hongyu Zhang. 2024. BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection. In *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*. Best Artifact Award; arXiv:2405.09330. doi:10.1145/3660805

Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data. In *Companion Proc. ACM Web Conference (WWW Companion)*. arXiv:2412.17015. doi:10.1145/3701716.3715290

Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hriţcu, Vilhelm Sjöberg, and Brent Yorgey. 2019. *Software Foundations*. Vol. 1: Logical Foundations. Electronic textbook. https://softwarefoundations.cis.upenn.edu/.

Gordon Plotkin and Matija Pretnar. 2009. Handlers of Algebraic Effects. In *Proc. European Symposium on Programming (ESOP)*. doi:10.1007/978-3-642-00590-9_7

Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring LLM-based Agents for Root Cause Analysis. In *Companion Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion, Industry Track)*. arXiv:2403.04123. doi:10.1145/3663529.3663841

Taro Sekiyama and Hiroshi Unno. 2025. Algebraic Temporal Effects: Temporal Verification of Recursively Typed Higher-Order Programs. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 78 (2025), 2306–2336 pages. Distinguished Paper Award. doi:10.1145/3704914

Manish Shetty, Yinfang Chen, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Xuchao Zhang, Jonathan Mace, Dax Vandevoorde, Pedro Las-Casas, Shachee Mishra Gupta, Suman Nath, Chetan Bansal, and Saravan Rajmohan. 2024. Building AI Agents for Autonomous Clouds: Challenges and Design Principles. In *Proc. ACM Symposium on Cloud Computing (SoCC)*. Vision paper; arXiv:2407.12165. Full system (AIOpsLab) appears as Chen et al., arXiv:2501.06706.. doi:10.1145/3698038.3698525

KC Sivaramakrishnan, Stephen Dolan, Leo White, Sadiq Jaffer, Anil Kelly, Anil Madhavapeddy, and Daan Leijen. 2021. Retrofitting Effect Handlers onto OCaml. In *Proc. ACM SIGPLAN Conference on Programming Language Design and*

Implementation (PLDI). doi:10.1145/3453483.3454039

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. doi:10.1145/1168917.1168907

Temporal Technologies. 2023. Temporal: Durable Execution Platform. https://temporal.io. Open-source workflow orchestration with durable execution and compensation.

Jesse A. Tov and Riccardo Pucella. 2011. Practical Affine Types. In *Proc. ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. doi:10.1145/1925844.1926436

Orpheas van Rooij and Robbert Krebbers. 2025. Affect: An Affine Type and Effect System. In *Proc. ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. Distinguished Paper Award. doi:10.1145/3704841

David Walker. 2005. Substructural Type Systems. In *Advanced Topics in Types and Programming Languages*, Benjamin C. Pierce (Ed.). MIT Press, Chapter 1.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. International Conference on Learning Representations (ICLR)*.

Takumi Yoshioka, Taro Sekiyama, and Atsushi Igarashi. 2024. Abstracting Effect Systems for Algebraic Effect Handlers. In *Proc. ACM SIGPLAN International Conference on Functional Programming (ICFP)*. arXiv:2404.16381. doi:10.1145/3674641

Zhaoyang Yu, Aoyang Fang, Minghua Ma, Jaskaran Singh Walia, Chaoyun Zhang, Shu Chi, Ze Li, Murali Chintalapati, Xuchao Zhang, Rujia Wang, Chetan Bansal, Saravan Rajmohan, Qingwei Lin, Shenglin Zhang, Dan Pei, and Pinjia He. 2025. Triangle: Empowering Incident Triage with Multi-Agent. In *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, and Ying Li. 2025a. A Survey of AIOps in the Era of Large Language Models. *Comput. Surveys* (2025). arXiv:2507.12472. doi:10.1145/3746635

Xiao Zhang, Qi Wang, Mingyi Li, Yuan Yuan, Mengbai Xiao, Fuzhen Zhuang, and Dongxiao Yu. 2025b. TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent with Multi-Modality Observation Data in Cloud-Native Systems. *arXiv preprint arXiv:2504.20462* (2025).

Received 2026-02-19; accepted 2026-05-13